
Documentação de Projeto

para o sistema

ReparoTech

Versão 1.0

Projeto de sistema elaborado pelo aluno Vincenzo Fonseca de Mello Souza
como parte da disciplina **Projeto de Software**.

21 de maio de 2026

Tabela de Conteúdo

1. Introdução	1
2. Modelos de Usuário e Requisitos	1
2.1 Descrição de Atores	1
2.1.1 Cliente	1
2.1.2 Funcionário (Técnico)	1
2.1.3 Gerente	2
2.1.4 Sistema de Pagamento	2
2.1.5 Diagrama de Atores	3
2.2 Modelo de Casos de Uso	3
2.2.1 Diagrama de Casos de Uso	3
2.2.2 Visão Geral dos Casos de Uso	4
Visão geral dos casos de uso	4
2.2.3 Detalhamento dos Casos de Uso	5
UC-01 — Cadastrar cliente	5
UC-02 — Cadastrar equipamento	5
UC-03 — Abrir ordem de serviço	6
UC-04 — Registrar diagnóstico técnico	6
UC-05 — Gerar orçamento	7
UC-06 — Aprovar ou recusar orçamento	7
UC-07 — Executar serviço técnico	7
UC-08 — Registrar uso de peças	8
UC-09 — Registrar pagamento	8
UC-10 — Finalizar ordem de serviço	9
UC-11 — Registrar retirada do equipamento	9
UC-12 — Gerenciar peças e estoque	9
UC-13 — Gerenciar serviços e preços	10
UC-14 — Gerenciar funcionários	10
UC-15 — Gerar relatórios gerenciais	10
2.2.4 Histórias de Usuário	11
2.3 Diagrama de Sequência do Sistema	12
2.3.1 Contratos de Operação	13
3. Modelos de Projeto	16
3.1 Arquitetura	16
3.1.1 Contexto e Sistemas Externos	16
3.1.2 Visão Lógica em Camadas	16

3.1.3 Decisões Arquiteturais	18
3.2 Diagrama de Componentes e Implantação.	18
3.2.1 Diagrama de Componentes	18
3.2.2 Diagrama de Implantação	19
3.3 Diagrama de Classes	20
3.3.1 Hierarquia de Usuário	20
3.3.2 Núcleo: OrdemServico	20
3.3.3 Orçamento e Itens	21
3.3.4 Estoque e Catálogo	21
3.3.5 Pagamento e Histórico	21
3.4 Diagramas de Sequência	22
3.4.1 Realização de UC-01 — Cadastrar Cliente	22
3.4.2 Realização de UC-02 — Cadastrar Equipamento	23
3.4.3 Realização de UC-03 — Abrir Ordem de Serviço	23
3.4.4 Realização de UC-04 — Registrar Diagnóstico Técnico	24
3.4.5 Realização de UC-05 — Gerar Orçamento	24
3.4.6 Realização de UC-06 — Aprovar ou Recusar Orçamento	25
3.4.7 Realização de UC-07 — Executar Serviço Técnico	25
3.4.8 Realização de UC-08 — Registrar Uso de Peças	26
3.4.9 Realização de UC-09 — Registrar Pagamento	27
3.4.10 Realização de UC-10 — Finalizar Ordem de Serviço	27
3.4.11 Realização de UC-11 — Registrar Retirada do Equipamento	28
3.4.12 Realização de UC-12 — Gerenciar Peças e Estoque	28
3.4.13 Realização de UC-13 — Gerenciar Serviços e Preços	28
3.4.14 Realização de UC-14 — Gerenciar Funcionários	29
3.4.15 Realização de UC-15 — Gerar Relatórios Gerenciais	29
3.5 Diagramas de Comunicação	30
3.6 Diagrama de Estados	32
3.6.1 Estados e organização	32
3.6.2 Transições e gatilhos	32
3.6.3 Registro histórico	33
4. Modelos de Dados	34
4.1 Esquema relacional	34
4.1.1 Índices não triviais	34
4.1.2 Restrição de integridade XOR em item_orcamento	34
4.2 Mapeamento objeto-relacional (Prisma)	34
4.3 Diagrama Entidade-Relacionamento	35

1. Introdução

Este documento agrega: (1) a elaboração e revisão dos modelos de domínio e (2) os modelos de projeto para o sistema **Reparo Tech**, uma aplicação web destinada à gestão de uma oficina de manutenção de computadores.

O Reparo Tech tem como objetivo organizar o ciclo completo de atendimento de equipamentos levados por clientes para serviços como formatação, troca de componentes, diagnóstico de problemas técnicos, remoção de vírus, limpeza preventiva, backup de dados e manutenção geral. O sistema controla ordens de serviço, clientes, equipamentos, orçamentos, peças, pagamentos e relatórios gerenciais.

Este projeto é desenvolvido em nível de **documentação, modelagem, arquitetura e diagramação**, sem implementação de código-fonte funcional. Todos os diagramas seguem o padrão PlantUML (e um Graphviz).

2. Modelos de Usuário e Requisitos

2.1 Descrição de Atores

Nesta subseção é apresentada a descrição de cada um dos atores que interagem com o sistema Reparo Tech. Os atores foram identificados a partir da análise dos processos da oficina e do escopo definido.

2.1.1 Cliente

- **Tipo:** Ator humano externo.
- **Descrição:** Pessoa física que leva um equipamento (notebook, desktop ou periférico) à oficina para manutenção. Não possui acesso direto ao sistema; interage indiretamente por meio de um funcionário no balcão de atendimento.
- **Principais interações:**
 - Fornecer dados cadastrais e do equipamento.
 - Descrever o problema apresentado pelo equipamento.
 - Aprovar ou recusar o orçamento gerado pela oficina.
 - Efetuar o pagamento ao final do serviço.
 - Retirar o equipamento após a finalização da ordem de serviço.
- **Conhecimento técnico esperado:** Nenhum.
- **Frequência de uso:** Eventual (por demanda de manutenção).

2.1.2 Funcionário (Técnico)

- **Tipo:** Ator humano interno.

- **Descrição:** Usuário operacional do sistema, responsável por atender o cliente, abrir e operar ordens de serviço, executar diagnósticos e serviços técnicos, registrar uso de peças e lançar pagamentos.

- **Principais interações:**

- Cadastrar clientes e equipamentos.
- Abrir ordens de serviço.
- Registrar diagnóstico técnico.
- Gerar orçamento.
- Registrar aprovação ou recusa do orçamento por parte do cliente.
- Executar serviços técnicos e registrar peças utilizadas.
- Registrar pagamento.
- Finalizar ordens de serviço e registrar retirada do equipamento.
- Consultar histórico de atendimentos.

- **Restrições de permissão:**

- Não pode alterar preços padrão de serviços e peças.
- Não pode gerenciar usuários ou permissões.
- Não pode emitir relatórios gerenciais consolidados.

- **Conhecimento técnico esperado:** Operacional em informática e procedimentos da oficina.

- **Frequência de uso:** Diária, ao longo de todo o expediente.

2.1.3 Gerente

- **Tipo:** Ator humano interno.

- **Descrição:** Usuário administrativo com permissões totais no sistema. Responsável pela gestão estratégica e operacional da oficina, incluindo equipe, catálogo de serviços, peças e indicadores de desempenho.

- **Principais interações:**

- Todas as operações disponíveis ao funcionário.
- Cadastrar, editar e desativar funcionários.
- Gerenciar catálogo de serviços e definir preços padrão.
- Gerenciar peças e estoque.
- Definir permissões de acesso.
- Gerar e analisar relatórios gerenciais (faturamento, produtividade técnica, peças mais utilizadas, tempo médio de atendimento).

- **Conhecimento técnico esperado:** Operacional em informática e gestão administrativa.

- **Frequência de uso:** Diária.

2.1.4 Sistema de Pagamento

- **Tipo:** Ator externo (sistema).

- **Descrição:** Ator conceitual que representa o serviço externo de processamento e confirmação de pagamentos (cartão de crédito, débito, PIX). No escopo deste projeto, a integração é apenas modelada, sem implementação real.

- **Principais interações:**

- Receber requisição de cobrança originada pelo Reparo Tech.

- Retornar o status do pagamento (Aprovado, Recusado ou Pendente).
- **Frequência de uso:** Sempre que uma ordem de serviço chega ao estágio de pagamento.

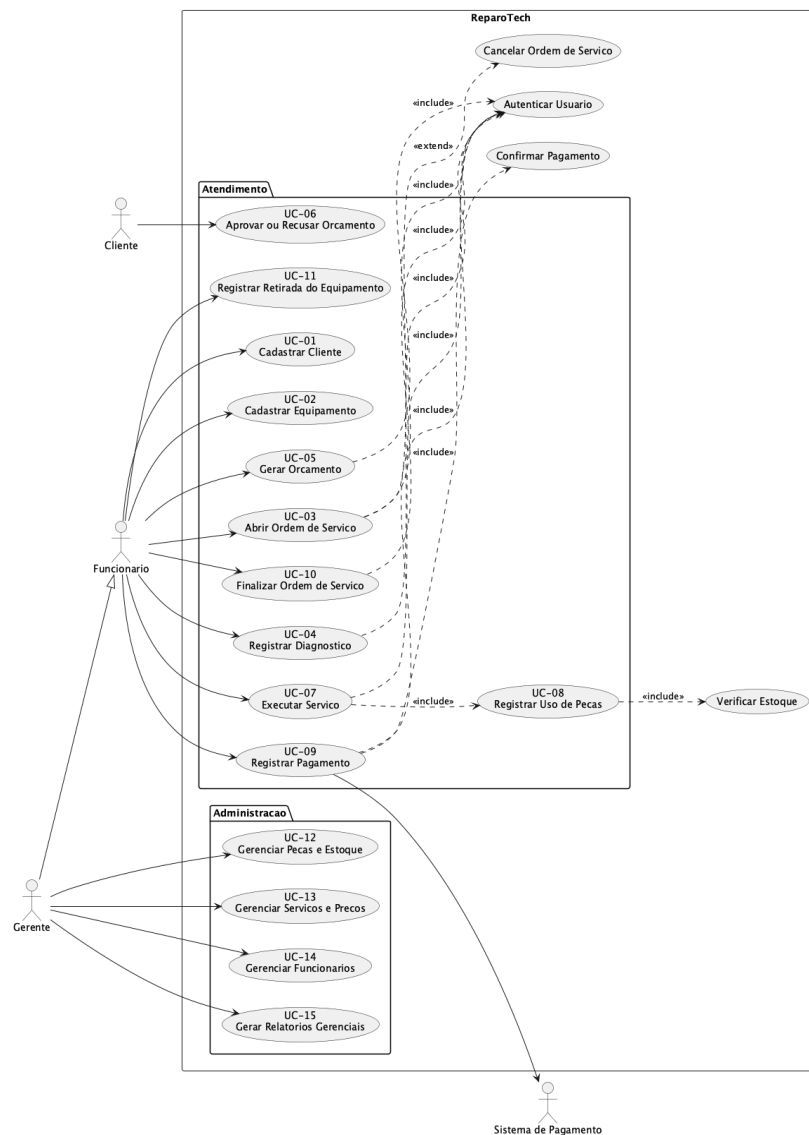
2.1.5 Diagrama de Atores

A representação visual dos atores e suas relações com os casos de uso está consolidada no diagrama de casos de uso (arquivo `01-casos-de-uso.puml`), inserido na Seção 2.2.1 como imagem.

2.2 Modelo de Casos de Uso

2.2.1 Diagrama de Casos de Uso

O diagrama geral de casos de uso do Reparo Tech (arquivo-fonte `01-casos-de-uso.puml`) relaciona os atores descritos na Seção 2.1 com os 15 casos de uso identificados, incluindo as relações de *include* e *extend* entre eles.



2.2.2 Visão Geral dos Casos de Uso

A tabela a seguir apresenta uma visão sintética dos casos de uso identificados para o Reparo Tech, relacionando cada funcionalidade ao seu ator principal e à categoria funcional correspondente. Essa visão geral serve como referência para o detalhamento dos fluxos descritos na Seção 2.2.3 e evidencia a divisão do sistema entre cadastros básicos, atendimento operacional e atividades administrativas.

Visão geral dos casos de uso			
ID	Nome	Ator Principal	Categoria
UC-01	Cadastrar cliente	Funcionário	Cadastros básicos
UC-02	Cadastrar equipamento	Funcionário	Cadastros básicos
UC-03	Abrir ordem de serviço	Funcionário	Atendimento
UC-04	Registrar diagnóstico técnico	Funcionário	Atendimento
UC-05	Gerar orçamento	Funcionário	Atendimento
UC-06	Aprovar ou recusar orçamento	Cliente via Funcionário	Atendimento
UC-07	Executar serviço técnico	Funcionário	Atendimento
UC-08	Registrar uso de peças	Funcionário	Atendimento
UC-09	Registrar pagamento	Funcionário	Atendimento
UC-10	Finalizar ordem de serviço	Funcionário	Atendimento
UC-11	Registrar retirada de equipamento	Funcionário	Atendimento
UC-12	Gerenciar peças e estoque	Gerente	Administração

Visão geral dos casos de uso			
UC-13	Gerenciar serviços e preços	Gerente	Administração
UC-14	Gerenciar funcionários	Gerente	Administração
UC-15	Gerar relatórios gerenciais	Gerente	Administração

2.2.3 Detalhamento dos Casos de Uso

UC-01 — Cadastrar cliente

- **Ator principal:** Funcionário.
- **Descrição:** Permite registrar um novo cliente no sistema com seus dados pessoais e de contato.
- **Pré-condições:** Funcionário autenticado no sistema.
- **Fluxo principal:**
 - O funcionário acessa o módulo de clientes.
 - O sistema exibe o formulário de cadastro.
 - O funcionário preenche nome, CPF, telefone, e-mail e endereço.
 - O funcionário confirma o cadastro.
 - O sistema valida os dados informados.
 - O sistema persiste o cliente e exibe mensagem de sucesso.
- **Fluxos alternativos:**
 - **A1 — CPF já cadastrado:** o sistema exibe alerta e oferece carregar o cliente existente.
- **Fluxos de exceção:**
 - **E1 — Dados obrigatórios ausentes:** o sistema bloqueia o salvamento e destaca os campos faltantes.
 - **E2 — CPF inválido:** o sistema impede o salvamento e solicita correção.
- **Pós-condições:** Cliente registrado e disponível para vinculação a equipamentos e ordens de serviço.
- **Regras de negócio relacionadas:** Identidade única do cliente (CPF único).

UC-02 — Cadastrar equipamento

- **Ator principal:** Funcionário.
- **Descrição:** Vincula um equipamento (notebook, desktop ou periférico) a um cliente já cadastrado.
- **Pré-condições:** Funcionário autenticado; cliente já cadastrado (UC-01).
- **Fluxo principal:**
 - O funcionário seleciona o cliente.
 - O sistema exibe o formulário de cadastro de equipamento.
 - O funcionário informa tipo, marca, modelo, número de série e observações.

- O funcionário confirma o cadastro.
- O sistema persiste o equipamento associado ao cliente.
- **Fluxos alternativos:**
- **A1 — Número de série já cadastrado:** o sistema oferece reutilizar o equipamento existente.
- **Fluxos de exceção:**
- **E1 — Cliente não selecionado:** o sistema impede o cadastro.
- **Pós-condições:** Equipamento vinculado ao cliente e disponível para abertura de ordem de serviço.
- **Regras de negócio:** Todo equipamento deve estar associado a um cliente.

UC-03 — Abrir ordem de serviço

- **Ator principal:** Funcionário.
- **Descrição:** Cria uma nova ordem de serviço para um equipamento, registrando o problema relatado pelo cliente.
- **Pré-condições:** Funcionário autenticado; cliente e equipamento já cadastrados.
- **Fluxo principal:**
- O funcionário seleciona o cliente e o equipamento.
- O sistema exibe o formulário de abertura de Ordem de Serviço.
- O funcionário registra o problema relatado e observações iniciais.
- O funcionário confirma a abertura.
- O sistema gera identificador único da Ordem de Serviço e define status **Aberta**.
- O sistema registra a entrada no histórico de status.
- **Fluxos alternativos:**
- **A1 — Cliente sem equipamento cadastrado:** o sistema direciona para UC-02.
- **Fluxos de exceção:**
- **E1 — Equipamento com Ordem de Serviço ativa:** o sistema alerta e impede abertura duplicada.
- **Pós-condições:** Ordem de Serviço criada com status **Aberta** e histórico inicial registrado.
- **Regras de negócio:** Uma Ordem de Serviço só pode ser aberta para cliente e equipamento cadastrados; status inicial obrigatoriamente **Aberta**.

UC-04 — Registrar diagnóstico técnico

- **Ator principal:** Funcionário (Técnico).
- **Descrição:** Registra o diagnóstico técnico realizado no equipamento da Ordem de Serviço.
- **Pré-condições:** Ordem de Serviço no status **Aberta**.
- **Fluxo principal:**
- O funcionário seleciona a Ordem de Serviço.
- O sistema altera o status para **Em Diagnóstico**.
- O funcionário registra a análise técnica, causa identificada e serviços/peças sugeridos.
- O funcionário confirma o diagnóstico.
- O sistema persiste o diagnóstico e atualiza status para **Aguardando Orçamento**.
- **Fluxos alternativos:**
- **A1 — Equipamento sem reparo viável:** o funcionário registra justificativa; status muda para **Aguardando Aprovação** com orçamento informativo.
- **Fluxos de exceção:**
- **E1 — Ordem de Serviço já com diagnóstico finalizado:** o sistema bloqueia novo registro (apenas edição autorizada).

- **Pós-condições:** Diagnóstico vinculado à Ordem de Serviço; status atualizado.
- **Regras de negócio:** O diagnóstico deve ser registrado antes da criação do orçamento.

UC-05 — Gerar orçamento

- **Ator principal:** Funcionário.
- **Descrição:** Monta o orçamento com base no diagnóstico, contendo serviços, peças e valor total.
- **Pré-condições:** Ordem de Serviço com diagnóstico registrado (UC-04).
- **Fluxo principal:**
 - O funcionário seleciona a Ordem de Serviço.
 - O sistema apresenta o catálogo de serviços e peças.
 - O funcionário adiciona os itens necessários ao orçamento.
 - O sistema calcula o valor total automaticamente.
 - O funcionário confirma a geração.
 - O sistema persiste o orçamento e atualiza status para **Aguardando Aprovação**.
- **Fluxos alternativos:**
 - **A1 — Orçamento sem peças:** apenas serviços são incluídos.
- **Fluxos de exceção:**
 - **E1 — Peça indisponível em estoque:** o sistema alerta e oferece marcar como peça a ser solicitada.
- **Pós-condições:** Orçamento criado e aguardando decisão do cliente.
- **Regras de negócio:** O orçamento deve conter serviços, peças (quando aplicável) e valor total.

UC-06 — Aprovar ou recusar orçamento

- **Ator principal:** Cliente (com Funcionário operando o sistema).
- **Descrição:** Registra a decisão do cliente sobre o orçamento apresentado.
- **Pré-condições:** Ordem de Serviço com status **Aguardando Aprovação**.
- **Fluxo principal:**
 - O funcionário apresenta o orçamento ao cliente.
 - O cliente comunica sua decisão.
 - O funcionário registra a decisão no sistema.
 - **Se aprovado:** o sistema atualiza status para **Orçamento Aprovado**.
 - **Se recusado:** o sistema atualiza status para **Orçamento Recusado** e oferece encerramento da Ordem de Serviço.
- **Fluxos alternativos:**
 - **A1 — Decisão postergada:** Ordem de Serviço permanece em **Aguardando Aprovação** com prazo registrado.
- **Fluxos de exceção:**
 - **E1 — Ordem de Serviço fora do status de aprovação:** o sistema impede o registro de decisão.
- **Pós-condições:** Decisão do cliente registrada e status atualizado.
- **Regras de negócio:** Caso o cliente recuse, a Ordem de Serviço pode ser encerrada sem execução do serviço.

UC-07 — Executar serviço técnico

- **Ator principal:** Funcionário (Técnico).
- **Descrição:** Executa os serviços previstos no orçamento aprovado.

- **Pré-condições:** Ordem de Serviço com status **Orçamento Aprovado**.
- **Fluxo principal:**
 - O funcionário inicia a execução; o sistema altera status para **Em Manutenção**.
 - O funcionário realiza os serviços e registra avanços.
 - Para cada peça utilizada, o funcionário aciona UC-08 (*include*).
 - Ao concluir, o funcionário marca o serviço como executado.
 - O sistema atualiza status para **Serviço Concluído**.
- **Fluxos alternativos:**
 - **A1 — Aguardando peça externa:** status muda para **Aguardando Peça**; retomada manual quando a peça chegar.
- **Fluxos de exceção:**
 - **E1 — Necessidade de orçamento adicional:** retorna para UC-05 com complemento de orçamento.
- **Pós-condições:** Serviços executados e peças devidamente abatidas do estoque.
- **Regras de negócio:** O serviço só pode ser executado após aprovação do orçamento.

UC-08 — Registrar uso de peças

- **Ator principal:** Funcionário.
- **Descrição:** Registra o consumo de peças durante a execução de uma Ordem de Serviço, atualizando o estoque.
- **Pré-condições:** Ordem de Serviço em execução (UC-07); peça disponível em estoque.
- **Fluxo principal:**
 - O funcionário seleciona a peça utilizada.
 - O sistema verifica disponibilidade (relação *include* com "Verificar Estoque").
 - O funcionário confirma a quantidade.
 - O sistema vincula a peça à Ordem de Serviço e debita o estoque.
- **Fluxos de exceção:**
 - **E1 — Peça sem disponibilidade:** o sistema bloqueia o uso e sugere solicitação ao gerente.
- **Pós-condições:** Peça vinculada à Ordem de Serviço; estoque atualizado.
- **Regras de negócio:** Não é permitido utilizar peça sem disponibilidade em estoque; peças utilizadas devem ser descontadas do estoque.

UC-09 — Registrar pagamento

- **Ator principal:** Funcionário.
- **Ator secundário:** Sistema de Pagamento.
- **Descrição:** Registra o pagamento da Ordem de Serviço pelo cliente.
- **Pré-condições:** Ordem de Serviço com status **Serviço Concluído** ou **Aguardando Pagamento**.
- **Fluxo principal:**
 - O funcionário seleciona a Ordem de Serviço.
 - O sistema apresenta o valor total devido.
 - O funcionário seleciona o método de pagamento.
 - O sistema envia requisição ao Sistema de Pagamento.
 - O Sistema de Pagamento retorna o status (**Aprovado**, **Recusado** ou **Pendente**).
 - O sistema registra o pagamento e atualiza o status da Ordem de Serviço conforme retorno.

- **Fluxos alternativos:**
- **A1 — Pagamento em dinheiro:** confirmação manual sem integração externa.
- **Fluxos de exceção:**
- **E1 — Pagamento recusado:** Ordem de Serviço permanece em Aguardando Pagamento.
- **E2 — Falha de comunicação com Sistema de Pagamento:** o sistema permite reprocessar a tentativa.
- **Pós-condições:** Pagamento registrado; status da Ordem de Serviço atualizado.
- **Regras de negócio:** Ordem de Serviço não pode ser finalizada sem pagamento confirmado, exceto em cancelamento ou recusa de orçamento.

UC-10 — Finalizar ordem de serviço

- **Ator principal:** Funcionário.
- **Descrição:** Encerra formalmente a Ordem de Serviço, deixando-a pronta para retirada.
- **Pré-condições:** Pagamento confirmado (UC-09) ou Ordem de Serviço recusada/cancelada.
- **Fluxo principal:**
- O funcionário seleciona a Ordem de Serviço.
- O sistema verifica as condições de finalização.
- O funcionário confirma a finalização.
- O sistema atualiza status para Finalizada.
- O sistema registra entrada no histórico.
- **Fluxos de exceção:**
- **E1 — Pagamento pendente:** o sistema impede a finalização.
- **Pós-condições:** Ordem de Serviço finalizada e disponível para retirada do equipamento.
- **Regras de negócio:** Ordem de Serviço não pode ser finalizada sem pagamento confirmado.

UC-11 — Registrar retirada do equipamento

- **Ator principal:** Funcionário.
- **Descrição:** Registra a retirada do equipamento pelo cliente após a finalização da Ordem de Serviço.
- **Pré-condições:** Ordem de Serviço com status Finalizada.
- **Fluxo principal:**
- O funcionário localiza a Ordem de Serviço pelo identificador.
- O sistema apresenta os dados de retirada.
- O funcionário confirma a retirada e registra a identificação de quem retirou.
- O sistema marca a Ordem de Serviço como retirada e arquiva o atendimento.
- **Fluxos de exceção:**
- **E1 — Ordem de Serviço não finalizada:** o sistema impede a retirada.
- **Pós-condições:** Equipamento entregue; ciclo de atendimento encerrado.
- **Regras de negócio:** O equipamento só pode ser entregue ao cliente após a Ordem de Serviço estar finalizada.

UC-12 — Gerenciar peças e estoque

- **Ator principal:** Gerente.
- **Descrição:** Permite cadastrar, editar, remover e reabastecer peças do estoque.
- **Pré-condições:** Gerente autenticado.

- **Fluxo principal:**

- O gerente acessa o módulo de estoque.
- O sistema lista as peças cadastradas com saldo atual.
- O gerente realiza a operação desejada (cadastro, edição, baixa ou entrada).
- O sistema atualiza o catálogo e o saldo do estoque.

- **Fluxos de exceção:**

- **E1 — Tentativa de remover peça vinculada a Ordem de Serviço ativa:** o sistema impede a remoção.

- **Pós-condições:** Catálogo e estoque atualizados.

- **Regras de negócio:** Apenas o gerente pode realizar essa operação.

UC-13 — Gerenciar serviços e preços

- **Ator principal:** Gerente.

- **Descrição:** Permite cadastrar, editar e definir preços padrão dos serviços oferecidos.

- **Pré-condições:** Gerente autenticado.

- **Fluxo principal:**

- O gerente acessa o catálogo de serviços.
- O sistema lista os serviços existentes.
- O gerente cadastra, edita ou desativa serviços e define seus preços padrão.
- O sistema persiste as alterações.
- **Pós-condições:** Catálogo de serviços atualizado e disponível para uso em orçamentos.
- **Regras de negócio:** Apenas o gerente pode alterar preços padrão.

UC-14 — Gerenciar funcionários

- **Ator principal:** Gerente.

- **Descrição:** Cadastra, edita ou desativa funcionários e define suas permissões.

- **Pré-condições:** Gerente autenticado.

- **Fluxo principal:**

- O gerente acessa o módulo de usuários.
- O sistema lista os funcionários cadastrados.
- O gerente cria, edita ou desativa um usuário e define o perfil de acesso.
- O sistema persiste as alterações.

- **Fluxos de exceção:**

- **E1 — Tentativa de desativar usuário com Ordem de Serviço em andamento:** o sistema alerta e exige reatribuição prévia.

- **Pós-condições:** Quadro de funcionários atualizado.

- **Regras de negócio:** Apenas o gerente pode gerenciar usuários e permissões.

UC-15 — Gerar relatórios gerenciais

- **Ator principal:** Gerente.

- **Descrição:** Gera relatórios consolidados sobre operação da oficina.

- **Pré-condições:** Gerente autenticado; existência de dados históricos.

- **Fluxo principal:**

- O gerente acessa o módulo de relatórios.

- O sistema apresenta os tipos disponíveis (faturamento, produtividade técnica, peças mais utilizadas, tempo médio de atendimento, Ordem de Serviço por status).
- O gerente seleciona o relatório e o intervalo de tempo.
- O sistema processa e exibe o relatório.
- O gerente pode exportar em PDF ou CSV.
- **Pós-condições:** Relatório gerado e disponível para análise/exportação.
- **Regras de negócio:** Apenas o gerente pode gerar relatórios gerenciais consolidados.

2.2.4 Histórias de Usuário

As histórias de usuário a seguir representam, de forma sintética, o valor entregue por cada caso de uso na perspectiva do usuário.

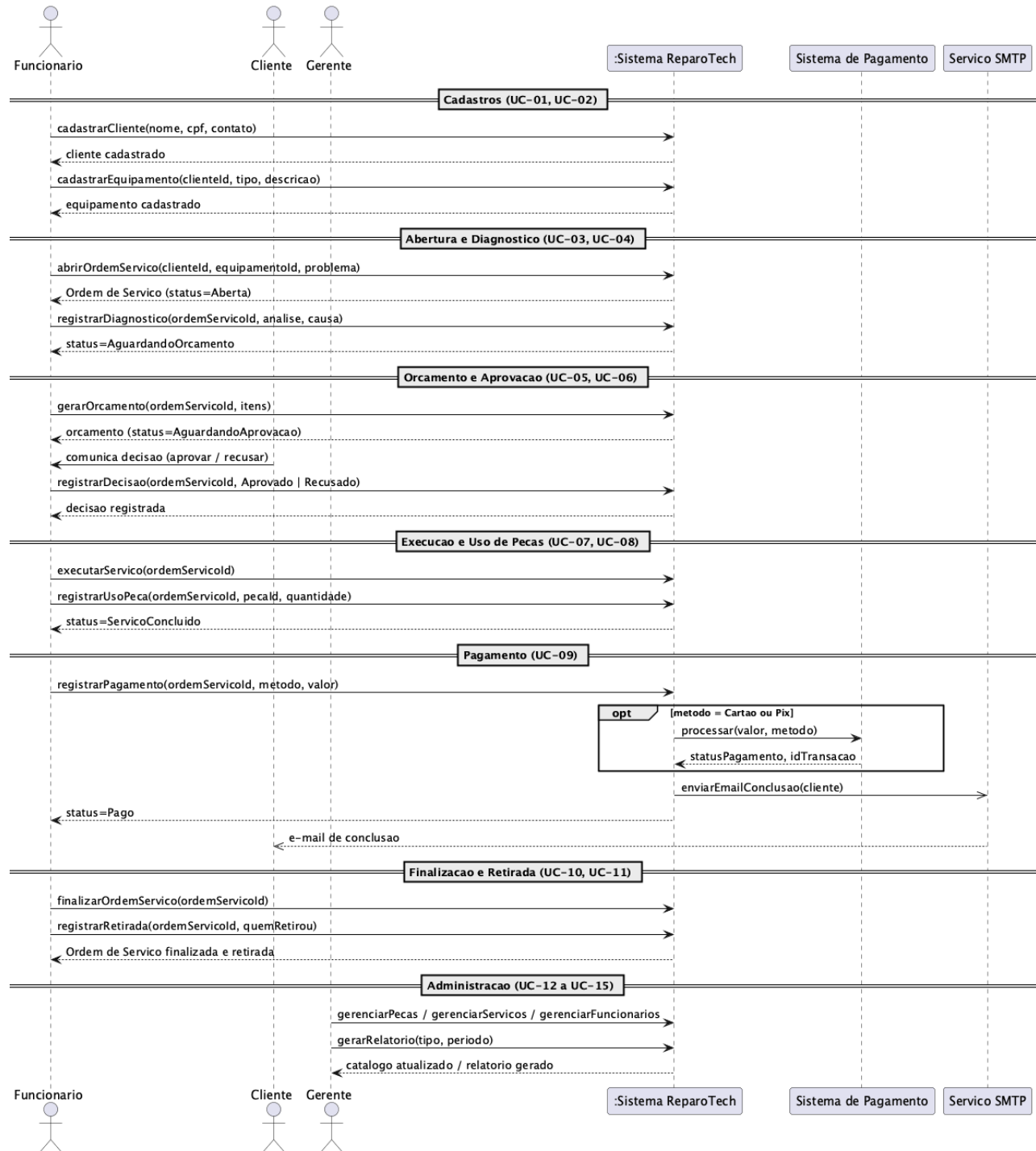
ID	História
HU-01	Como funcionário , quero cadastrar clientes , para vincular suas informações às ordens de serviço .
HU-02	Como funcionário , quero cadastrar equipamentos do cliente , para identificar com precisão o item recebido .
HU-03	Como funcionário , quero abrir uma ordem de serviço , para iniciar formalmente o atendimento .
HU-04	Como técnico , quero registrar o diagnóstico , para fundamentar o orçamento ao cliente .
HU-05	Como funcionário , quero gerar um orçamento , para apresentar serviços, peças e valor total ao cliente .
HU-06	Como cliente , quero aprovar ou recusar o orçamento , para decidir sobre a execução do serviço .
HU-07	Como técnico , quero executar o serviço , para resolver o problema do equipamento .
HU-08	Como funcionário , quero registrar o uso de peças , para manter o estoque atualizado .
HU-09	Como funcionário , quero registrar o pagamento , para encerrar a parte financeira do atendimento .
HU-10	Como funcionário , quero finalizar a ordem de serviço , para deixá-la pronta para retirada .

HU-11	Como funcionário , quero registrar a retirada do equipamento , para encerrar o ciclo de atendimento .
HU-12	Como gerente , quero gerenciar peças e estoque , para garantir disponibilidade dos itens utilizados .
HU-13	Como gerente , quero gerenciar serviços e preços , para manter o catálogo coerente com a política comercial .
HU-14	Como gerente , quero gerenciar funcionários , para controlar acesso e responsabilidades na oficina .
HU-15	Como gerente , quero gerar relatórios gerenciais , para acompanhar desempenho e tomar decisões baseadas em dados .

2.3 Diagrama de Sequência do Sistema

Nesta subseção é apresentado o Diagrama de Sequência do Sistema (DSS) do Reparo Tech em uma visão panorâmica única, que percorre o fluxo completo do sistema organizado em fases, uma para cada etapa do ciclo, do cadastro inicial à retirada do equipamento, incluindo o pagamento e as operações administrativas (UC-01 a UC-15).

Trata-se de uma visão caixa-preta: o Reparo Tech aparece como um único participante (:Sistema ReparoTech), com o qual os atores (Funcionário, Cliente e Gerente) interagem por meio de operações de sistema, sem expor a arquitetura interna. Apenas os dois sistemas genuinamente externos: Sistema de Pagamento e Serviço SMTP, são representados como participantes à parte. O detalhamento da colaboração interna (Controllers, Services, Repositories e Prisma) fica a cargo dos diagramas de sequência da Seção 3.4. O arquivo-fonte é 02-sequencia-de-sistema.puml.



2.3.1 Contratos de Operação

Foram selecionadas as operações centrais do ciclo, cujas pré-condições e pós-condições materializam regras de negócio do sistema. Operações triviais (consultas e CRUDs simples) foram omitidas para evitar redundância.

Contrato CO-1: abrirOrdemServico

Contrato	CO-1
Operação	<code>abrirOrdemServico(clienteId, equipamentoId, problemaRelatado): ordemServicoDTO</code>
Referências cruzadas	UC-03; UC-01 (cliente cadastrado); UC-02 (equipamento cadastrado). Regras: Ordem de Serviço só pode ser aberta para cliente e equipamento cadastrados; toda Ordem de Serviço inicia com status Aberta; toda mudança de status é registrada no histórico.
Pré-condições	Funcionário autenticado. Cliente identificado por <code>clienteId</code> existe. Equipamento identificado por <code>equipamentoId</code> existe e pertence ao cliente. Não há Ordem de Serviço ativa em andamento para esse equipamento.
Pós-condições	Nova Ordem de Serviço criada com identificador único. Status da Ordem de Serviço = Aberta. Ordem de Serviço vinculada ao cliente e ao equipamento. Problema relatado registrado. Entrada inicial gravada no histórico de status com timestamp e funcionário responsável.

Contrato CO-2: adicionarItemOrcamento

Contrato	CO-2
Operação	<code>adicionarItemOrcamento(ordemServicoId, tipo, itemId, quantidade): itemOrcamentoDTO</code>
Referências cruzadas	UC-05; UC-08 (registrar uso de peças). Regras: orçamento deve conter serviços e peças quando aplicável; não é permitido utilizar peça sem disponibilidade em estoque.
Pré-condições	Funcionário autenticado. Ordem de Serviço identificada por <code>ordemServicoId</code> existe e está no status Aguardando Orçamento. Item identificado por <code>itemId</code> existe no catálogo correspondente ao <code>tipo</code> (Serviço ou Peça). Quantidade maior que zero. Se <code>tipo</code> = Peça, saldo de estoque $\geq$ quantidade (caso contrário, fluxo alternativo trata o item como peça a solicitar).

Pós-condições	Item de orçamento criado e vinculado à Ordem de Serviço. Subtotal do item calculado como preço unitário × quantidade. Valor total provisório do orçamento atualizado. Se tipo = Peça, a quantidade fica reservada (a baixa efetiva no estoque ocorre apenas em UC-08, durante a execução).
----------------------	--

Contrato CO-3: confirmarOrçamento

Contrato	CO-3
Operação	confirmarOrçamento(ordemServicoId) : orcamentoDTO
Referências cruzadas	UC-05; UC-04 (diagnóstico técnico); UC-06 (aprovação/recusa). Regras: diagnóstico deve ser registrado antes da criação do orçamento; orçamento deve conter valor total.
Pré-condições	Funcionário autenticado. Ordem de Serviço identificada por ordemServicoId existe e está no status Aguardando Orçamento. Diagnóstico técnico (UC-04) já registrado na Ordem de Serviço. Pelo menos um item de orçamento adicionado.
Pós-condições	Orçamento da Ordem de Serviço é encerrado e o valor total final calculado. Status da Ordem de Serviço muda para Aguardando Aprovação. Entrada registrada no histórico de status. Orçamento disponível para apreciação do cliente (UC-06).

Contrato CO-4: registrarPagamento

Contrato	CO-4
Operação	registrarPagamento(ordemServicoId, metodo, valor): pagamentoDTO
Referências cruzadas	UC-09; UC-10 (finalização). Regras: Ordem de Serviço não pode ser finalizada sem pagamento confirmado, exceto em cancelamento ou recusa de orçamento; integração com Sistema de Pagamento é apenas modelada.
Pré-condições	Funcionário autenticado. Ordem de Serviço identificada por ordemServicoId existe e está no status Serviço Concluído ou

	Aguardando Pagamento. Valor informado $\geq$ valor total da Ordem de Serviço. Método de pagamento válido (dinheiro, cartão de crédito, cartão de débito ou PIX).
Pós-condições	Tentativa de pagamento registrada com status retornado pelo Sistema de Pagamento (Aprovado, Recusado ou Pendente). Se Aprovado, a Ordem de Serviço torna-se elegível para finalização (UC-10) e o cliente é notificado de forma assíncrona. Se Recusado, a Ordem de Serviço permanece em Aguardando Pagamento e o motivo é registrado. Se Pendente, o sistema agenda consultas periódicas ao Sistema de Pagamento até a confirmação. Entrada registrada no histórico de status.

3. Modelos de Projeto

3.1 Arquitetura

A arquitetura do Reparo Tech é apresentada em um único diagrama em camadas (arquivo-fonte 03-arquitetura.puml), que combina, numa só imagem, o contexto do sistema: atores (Cliente, Funcionário e Gerente) e sistemas externos (Sistema de Pagamento e Serviço SMTP) com a organização interna segundo o estilo arquitetural Layered. O diagrama traz legenda e ícones de ator, facilitando a leitura.

3.1.1 Contexto e Sistemas Externos

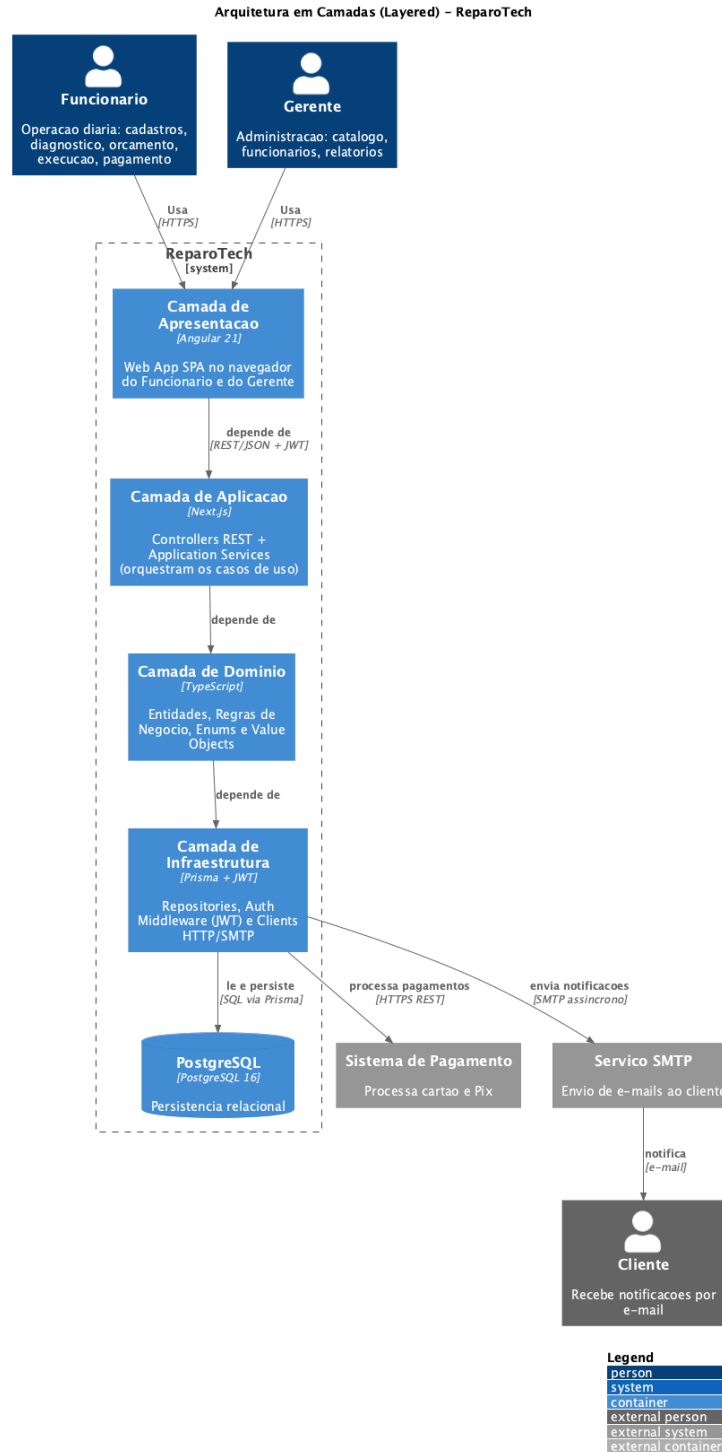
No nível de contexto, o Reparo Tech relaciona-se com três atores humanos: Cliente, Funcionário e Gerente, e com dois sistemas externos: o Sistema de Pagamento e o Serviço SMTP. Funcionário e Gerente usam o sistema diretamente pelo navegador (camada de Apresentação), enquanto o Cliente é receptor passivo, recebendo notificações por e-mail. Esses elementos aparecem em torno do sistema, no mesmo diagrama em camadas.

Funcionário e Gerente utilizam o sistema diretamente por meio do navegador, acessando o Frontend (Angular 21). O Frontend consome o Backend (Next.js + Prisma) por REST autenticado com JWT sobre HTTPS. O Backend persiste e consulta dados no PostgreSQL via Prisma e integra-se de forma síncrona ao Sistema de Pagamento e de forma assíncrona ao Serviço SMTP, que entrega as notificações por e-mail ao Cliente, receptor passivo da interação.

3.1.2 Visão Lógica em Camadas

A organização interna segue o estilo arquitetural Layered clássico, com quatro camadas representadas como contêineres empilhados e dependência unidirecional descendente, ou seja, cada camada depende apenas da imediatamente inferior.

A escolha pelo *Layered* clássico reflete o porte do projeto e a complexidade do domínio: a dependência unidirecional Apresentação → Aplicação → Domínio → Infraestrutura é mais simples de ensinar, documentar e implementar, sem prejuízo à organização do código.



3.1.3 Decisões Arquiteturais

- **Separação cliente/servidor:** o Angular roda no navegador e a Next.js atua apenas como API, permitindo evolução independente de front-end e back-end.
- **Autenticação stateless (JWT):** elimina necessidade de sessões no servidor; cada requisição carrega o token, simplificando o escalonamento horizontal.
- **PostgreSQL com Prisma:** banco relacional escolhido pela natureza fortemente normalizada do domínio (clientes, Ordens de Serviço, orçamentos e peças, com relações 1-N e N-N); Prisma fornece tipagem e migrações versionadas.
- **Integrações externas restritas:** apenas Sistema de Pagamento e Serviço SMTP; demais funções (estoque, relatórios) ficam dentro da própria API para simplificar o escopo conceitual.
- **Docker no deploy:** cada subsistema interno (Frontend, Backend, PostgreSQL) corresponde a um container Docker no diagrama de implantação (Seção 3.2), favorecendo a reprodutibilidade do ambiente.

Camada	Conteúdo	Responsabilidade
Apresentação	Web App (Angular 21)	Interface gráfica executada no navegador do Funcionário e do Gerente; consome a API.
Aplicação	Controllers REST, Application Services	Recebe requisições da camada de Apresentação, orquestra os casos de uso e dispara as regras de negócio.
Domínio	Entidades, Regras de Negócio, Enums e Value Objects	Modelo central; mantém o estado e a lógica de negócio do Reparo Tech, independente de tecnologia.
Infraestrutura	Repositories (Prisma), Auth Middleware (JWT), Clients HTTP/SMTP	Implementação dos detalhes técnicos: persistência no PostgreSQL, autenticação e integrações com sistemas externos.

3.2 Diagrama de Componentes e Implantação.

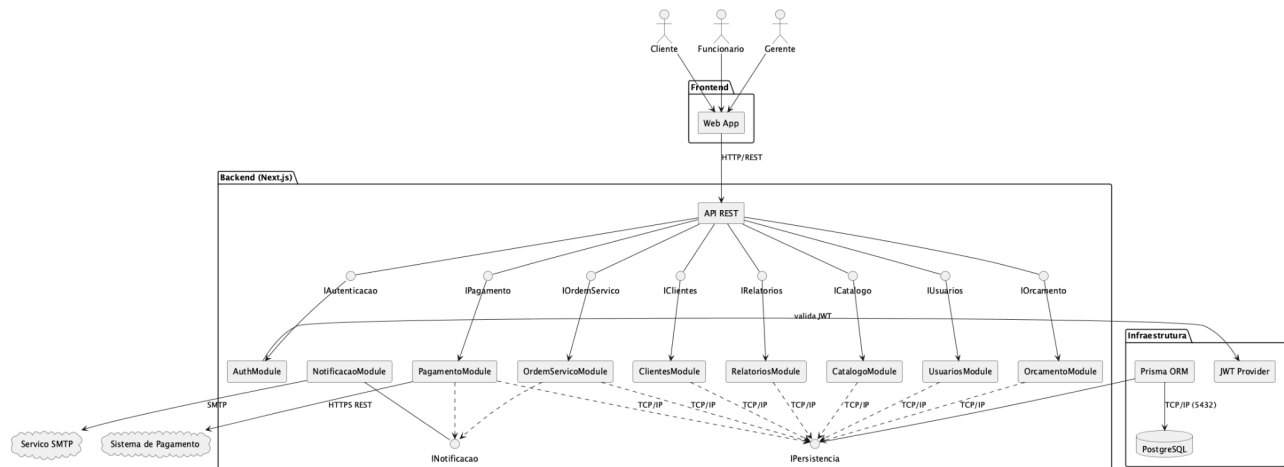
Esta subseção refina a arquitetura apresentada na Seção 3.1 sob duas perspectivas complementares: a estrutura interna do contêiner API (diagrama de componentes) e a topologia física de execução (diagrama de implantação).

3.2.1 Diagrama de Componentes

O contêiner API é decomposto em nove módulos de domínio, mais o ORM Prisma, que atua como camada técnica de persistência compartilhada. A decomposição segue a seguinte dinâmica: cada módulo agrupa as responsabilidades de uma entidade ou subdomínio do Reparo Tech. O diagrama está no arquivo-fonte `04-componentes.puml`.

O API REST expõe cada módulo por uma interface (notação lollipop) e delega a ele o tratamento da requisição. As colaborações de domínio existem conceitualmente, por exemplo, um orçamento sempre pertence a uma Ordem de Serviço e consulta o catálogo de itens; o pagamento, após confirmado, aciona o módulo de notificação. No diagrama, destacam-se as dependências para o `NotificacaoModule` (consumido por `OrdemServicoModule` e `PagamentoModule`) e a persistência compartilhada: todos os módulos com estado próprio acessam o banco por uma interface `IPersistencia` provida pelo Prisma ORM, que se comunica com o PostgreSQL.

Duas integrações externas são representadas: o `PagamentoModule` chama o Sistema de Pagamento por HTTPS REST e o `NotificacaoModule` envia e-mails ao Serviço SMTP externo.



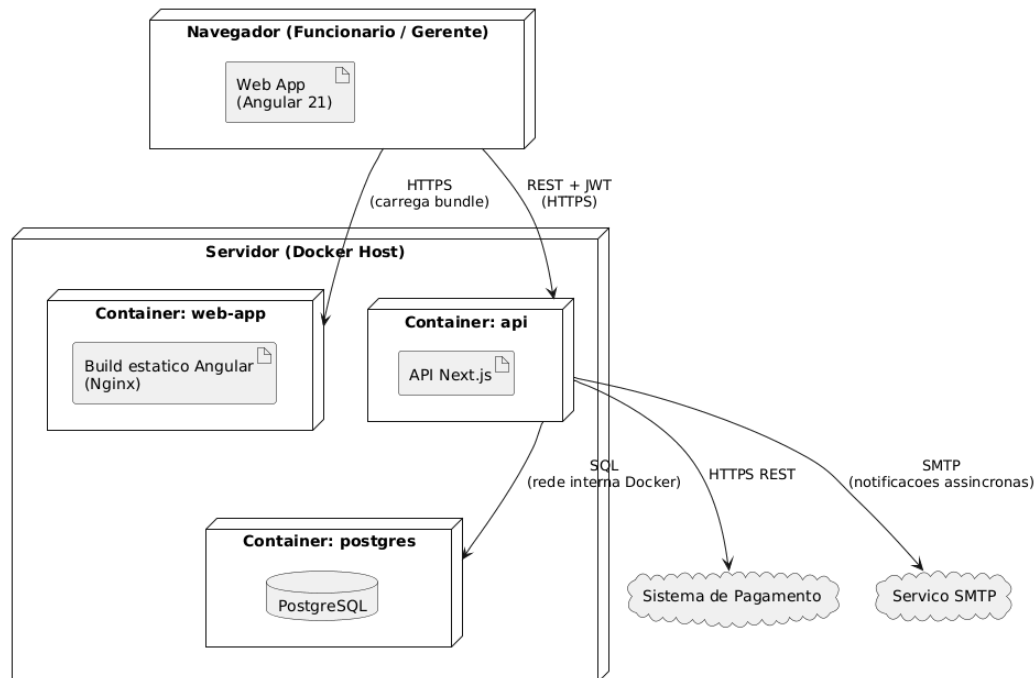
3.2.2 Diagrama de Implantação

O diagrama de implantação mostra onde cada contêiner do Reparo Tech roda fisicamente em um ambiente conceitual baseado em Docker. O arquivo-fonte é `05-implantacao.puml`.

Ele é composto por:

- **Navegador do Funcionário / Gerente:** nó cliente onde o Web App em Angular 21 é executado. Carrega o bundle estático servido pelo container `web-app` e consome a API via HTTPS.
- **Servidor (Docker Host):** nó servidor que hospeda três containers Docker:
- `web-app`: serve o build estático Angular por meio de um servidor HTTP leve (Nginx).
- `api`: instância da API Next.js, exposta por HTTPS.
- `postgres`: instância do PostgreSQL, acessível apenas pela rede interna do Docker (não exposta publicamente).
- **Sistema de Pagamento:** nó externo (cloud) acionado pelo container `api` por HTTPS REST.
- **Serviço SMTP:** nó externo (cloud) que recebe as solicitações de envio de e-mail do container `api` e entrega as notificações aos clientes.

Esse modelo conceitual deixa claro que a separação entre interface, backend e banco é também física (em containers distintos), favorecendo a reprodutibilidade do ambiente e o isolamento de falhas. Em produção real, cada container poderia ser realocado para hosts ou serviços gerenciados distintos sem mudança no código.



3.3 Diagrama de Classes

O diagrama de classes representa a estrutura estática do domínio do Reparo Tech: as entidades de negócio, seus atributos, seus métodos essenciais e os relacionamentos entre elas. O arquivo-fonte está em `06-classes.puml`.

Foram modeladas 13 classes mais cinco enumerações, agrupadas em três grupos:

- **Usuários e clientes:** Usuario (abstrata), Funcionario, Gerente e Cliente.
- **Núcleo de atendimento:** Equipamento, OrdemServico, Diagnostico, HistoricoStatus.
- **Comercial e financeiro:** Orcamento, ItemOrcamento, Servico, Peca, Pagamento.

3.3.1 Hierarquia de Usuário

Usuario é uma classe abstrata que centraliza atributos de autenticação (email, senha, ativo) e o método autenticar(senha). Suas duas especializações são Funcionario (com atributo matricula) e Gerente. A distinção de permissões é tratada nos módulos da API: operações administrativas (relatórios, gestão de preços, gestão de funcionários) exigem instância de Gerente.

3.3.2 Núcleo: OrdemServico

OrdemServico é a classe central do modelo. Conhece o Cliente, o Equipamento, o funcionário responsável, e agrega por composição (relacionamento parte-todo) instâncias de Diagnostico, Orcamento e HistoricoStatus. Possui o atributo status do tipo StatusOrdemServico e o método alterarStatus(novo) que aplica as regras de transição

entre os 12 estados (modelados detalhadamente na Seção 3.6). Cada chamada de `alterarStatus` gera uma entrada de `HistoricoStatus` correspondente.

3.3.3 Orçamento e Itens

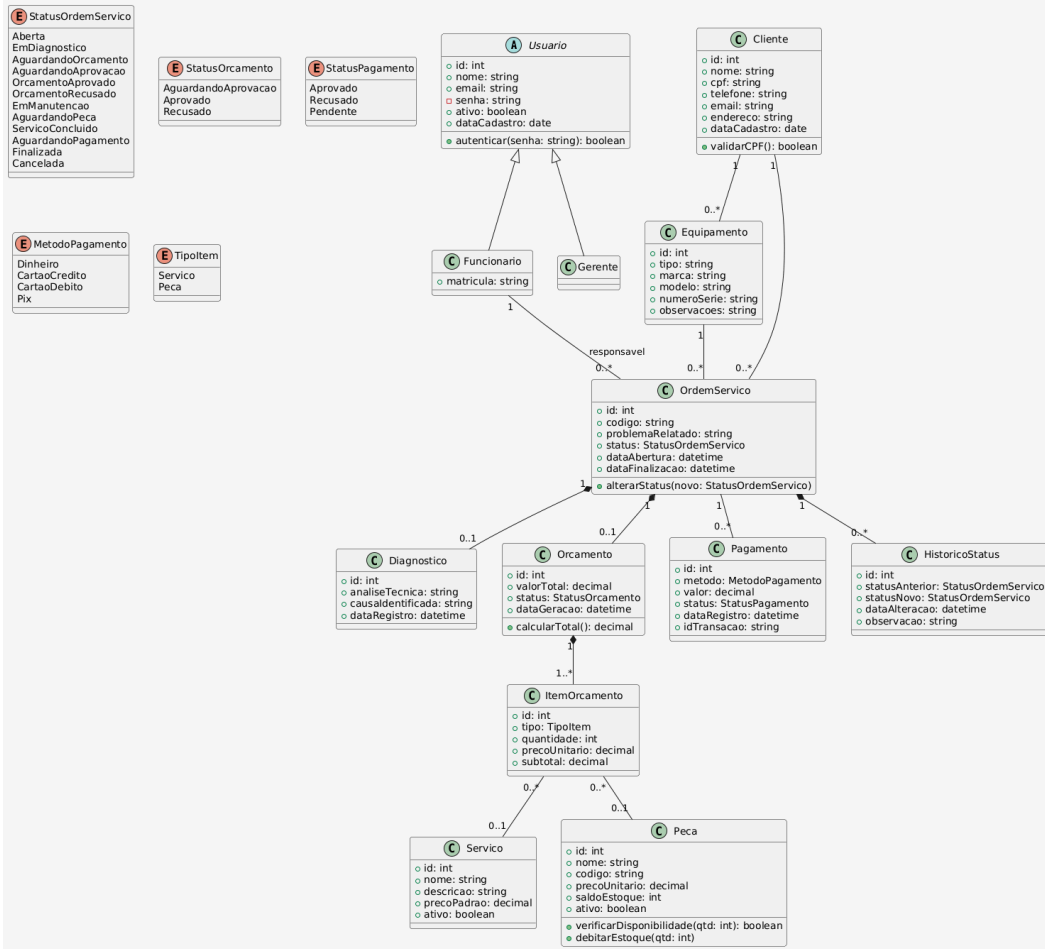
`Orcamento` agrega por composição uma lista de `ItemOrcamento`. Cada item pode referenciar tanto um `Servico` quanto uma `Peca`, conforme o atributo `tipo` (do enum `TipoItem`). O método `calcularTotal()` consolida os subtotais dos itens. O fluxo de aprovação/recusa do orçamento é representado pelo atributo `status` (do tipo `StatusOrcamento`).

3.3.4 Estoque e Catálogo

`Peca` carrega o atributo `saldoEstoque` e os métodos `verificarDisponibilidade(qtd)` e `debitarEstoque(qtd)`, que materializam as regras de negócio "não utilizar peça sem disponibilidade" e "peças utilizadas devem ser descontadas do estoque". `Servico` é uma classe simples de catálogo, com `precoPadrao` definido pelo gerente.

3.3.5 Pagamento e Histórico

`Pagamento` registra cada tentativa de cobrança vinculada à `OrdemServico`, com método (`MetodoPagamento`), valor, status e identificador externo da transação. Como uma mesma `OrdemServico` pode ter múltiplas tentativas (ex.: cartão recusado, seguido de PIX aprovado), a cardinalidade é `1..*`. `HistoricoStatus` registra cada mudança de status da `OrdemServico` para fins de auditoria, atendendo à regra "toda mudança de status deve ser registrada no histórico".



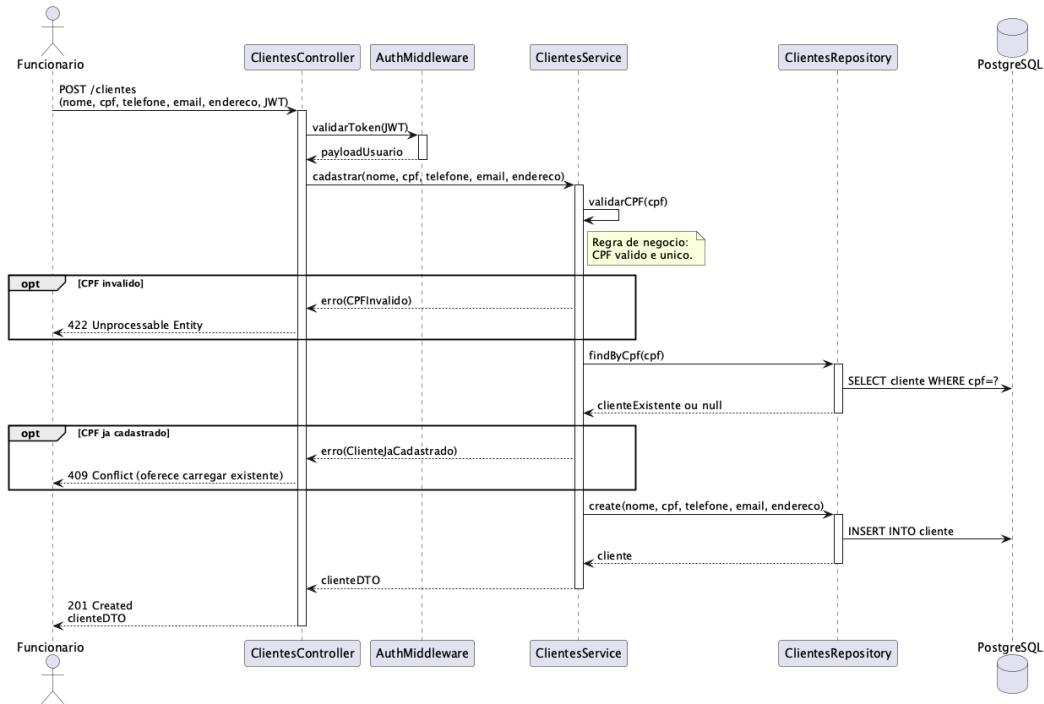
3.4 Diagramas de Sequência

Esta subseção apresenta os diagramas de sequência detalhados (caixa-branca) que realizam internamente cada caso de uso do sistema. Enquanto o Diagrama de Sequência do Sistema da Seção 2.3 trata o Reparo Tech como um único participante, aqui é mostrada a colaboração entre os módulos da API e as classes do modelo de domínio.

É apresentado um diagrama para cada um dos quinze casos de uso (UC-01 a UC-15), todos atravessando as camadas Controller, AuthMiddleware, Service, Repository e Prisma até o PostgreSQL. Os arquivos-fonte estão na pasta docs/diagramas-puml/07-sequencia-de-projeto/, com um arquivo .puml por caso de uso (seq-uc01-... a seq-uc15-...).

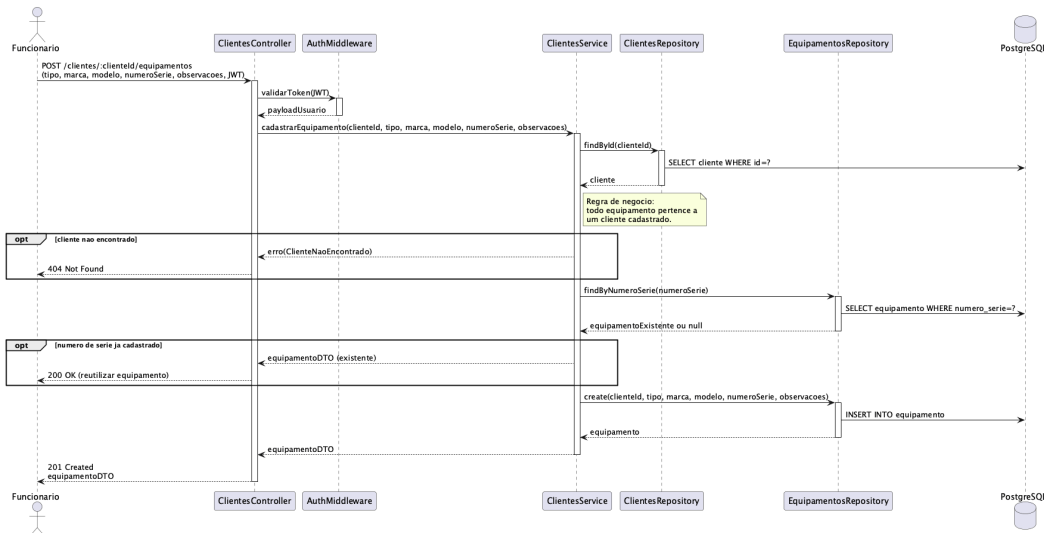
3.4.1 Realização de UC-01 — Cadastrar Cliente

A requisição POST /clientes é autenticada pelo AuthMiddleware e encaminhada ao ClientesController, que delega ao ClientesService. O serviço valida o CPF (formato e unicidade via ClientesRepository) antes de persistir o novo cliente no PostgreSQL.



3.4.2 Realização de UC-02 — Cadastrar Equipamento

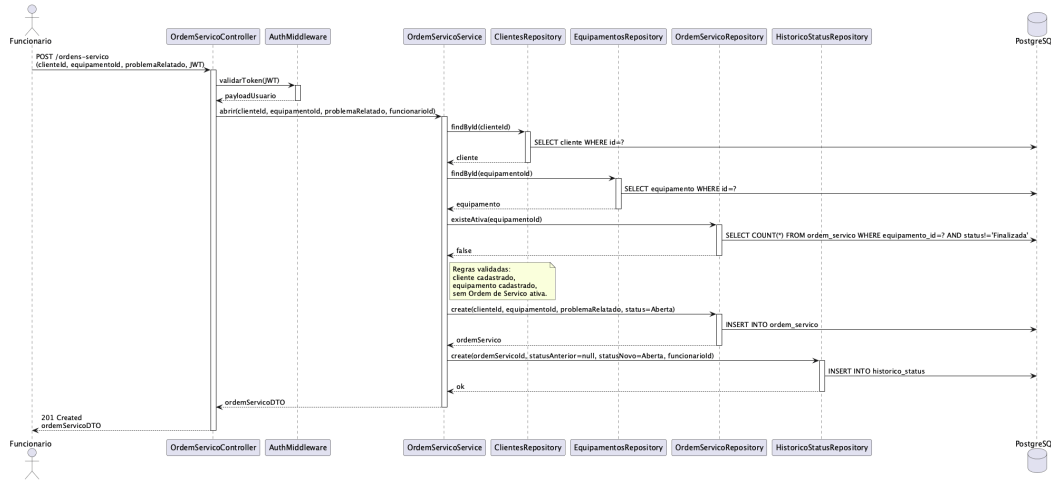
POST /clientes/:clienteId/equipamentos chega ao **ClientesController** e segue para o **ClientesService**, que confirma a existência do cliente (**ClientesRepository**) e checa o número de série antes de criar o equipamento via **EquipamentosRepository**, vinculando-o ao cliente.



3.4.3 Realização de UC-03 — Abrir Ordem de Serviço

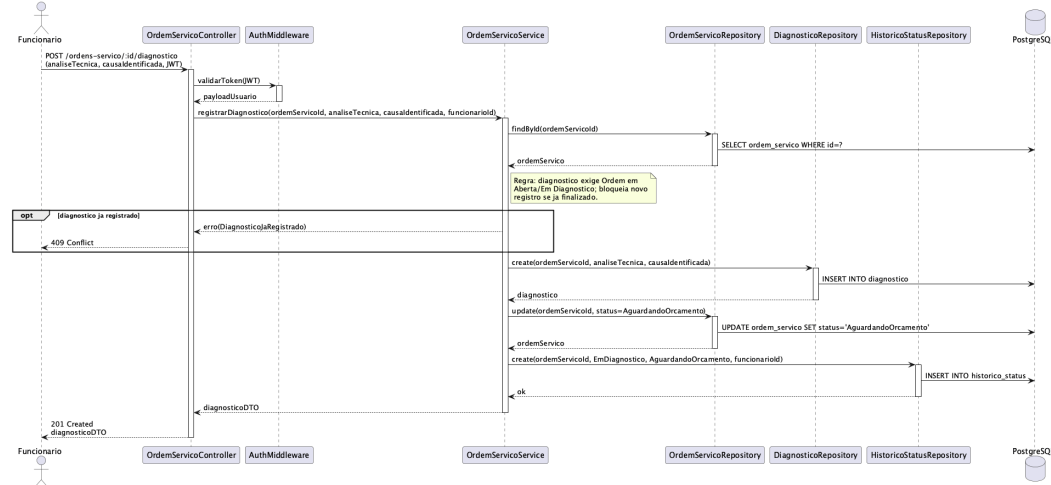
Mostra como a requisição **POST /ordens-servico** percorre o **OrdemServicoController**, é autenticada pelo **AuthMiddleware**, segue para o **OrdemServicoService**, que coordena consultas a **ClientesRepository**, **EquipamentosRepository** e ao próprio **OrdemServicoRepository** (para validar a

unicidade da Ordem de Serviço ativa), executa o INSERT da nova Ordem de Serviço e, em seguida, registra a entrada inicial em HistoricoStatusRepository. Em todas as escritas, o Prisma realiza a persistência no PostgreSQL.



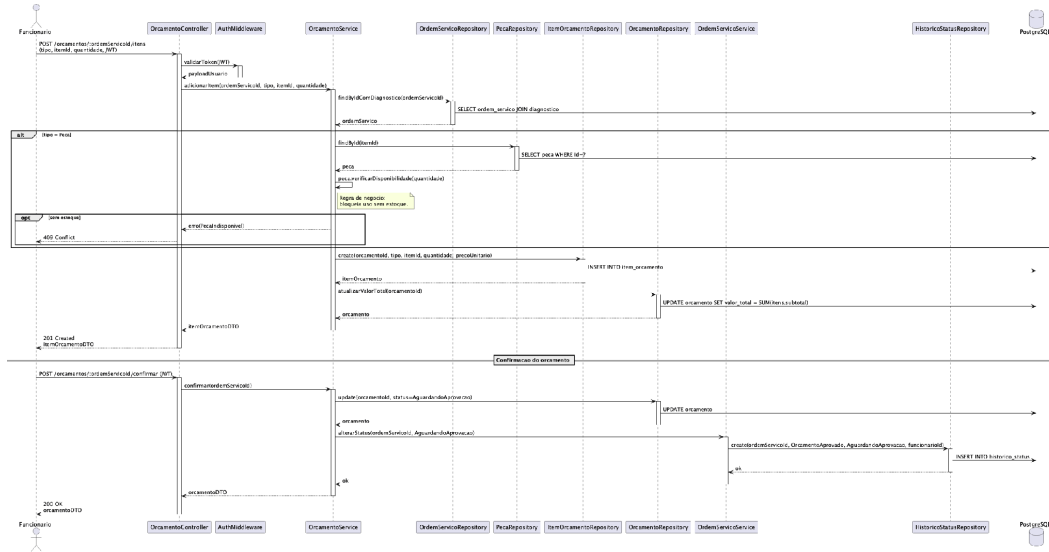
3.4.4 Realização de UC-04 — Registrar Diagnóstico Técnico

O OrdemServicoController recebe POST /ordens-servico/:id/diagnostico e aciona o OrdemServicoService, que cria o Diagnóstico (DiagnosticoRepository), atualiza o status da Ordem de Serviço para Aguardando Orçamento e registra a transição em HistoricoStatusRepository.



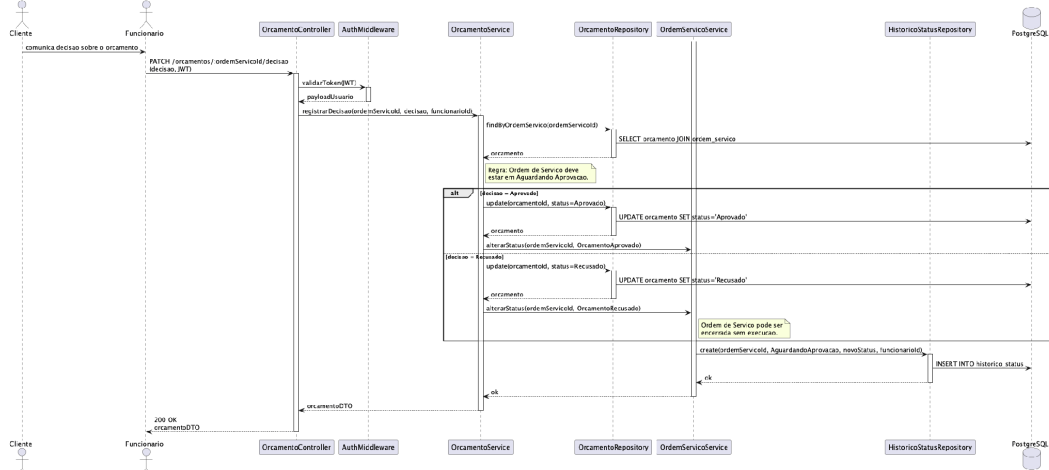
3.4.5 Realização de UC-05 — Gerar Orçamento

O diagrama é composto por dois fragmentos. O primeiro modela a adição iterativa de itens ao orçamento: OrcamentoService valida a existência da Ordem de Serviço com diagnóstico registrado, consulta o catálogo apropriado (PecaRepository ou serviços), aciona verificarDisponibilidade quando o item é uma peça e persiste o ItemOrcamento com recálculo do valor total. O segundo fragmento (separado pelo divisor Confirmacao do orcamento) modela a confirmação final, que delega ao OrdemServicoService.alterarStatus a transição para Aguardando Aprovação e o consequente registro no HistoricoStatus.



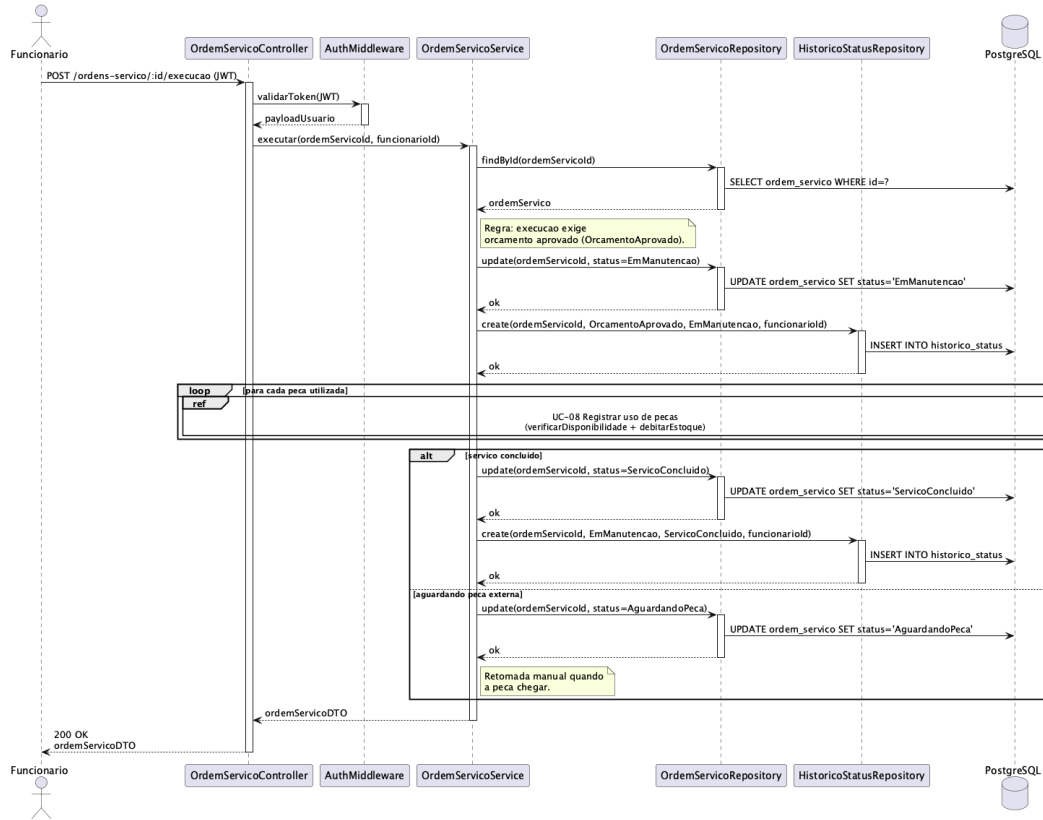
3.4.6 Realização de UC-06 — Aprovar ou Recusar Orçamento

Registra a decisão do cliente (operada pelo funcionário) via PATCH /orcamentos/:id/decisao. O OrcamentoService atualiza o status do orçamento (Aprovado ou Recusado) e delega ao OrdemServicoService a transição de estado correspondente, com registro no HistoricoStatus.



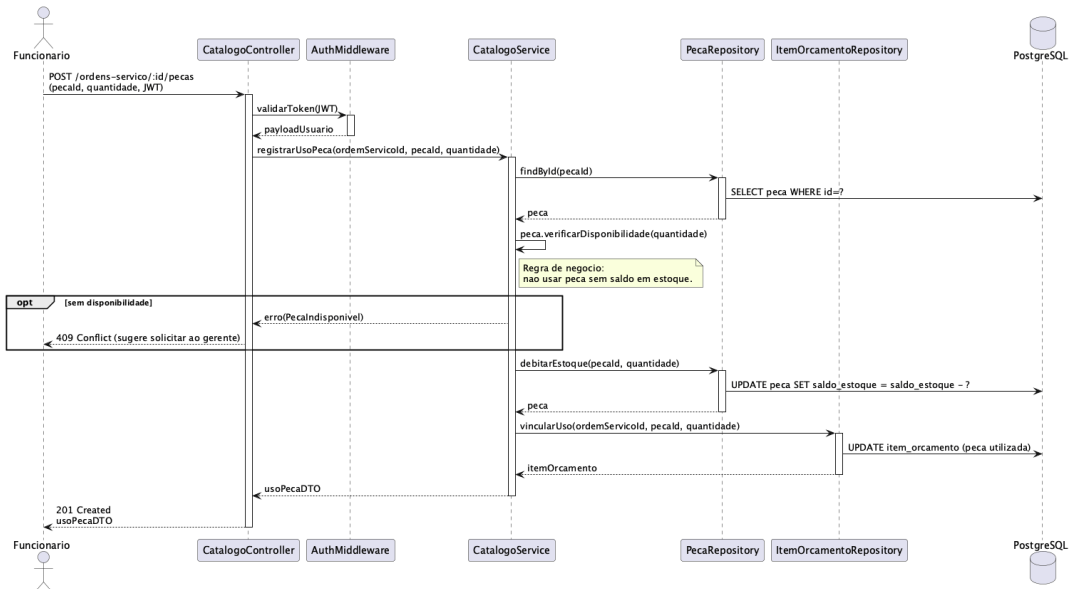
3.4.7 Realização de UC-07 — Executar Serviço Técnico

Inicia a execução do serviço: o OrdemServicoService altera o status para Em Manutenção e, para cada peça utilizada, aciona o UC-08 (relação «include»). Ao final, conclui em Serviço Concluído ou, faltando peça, em Aguardando Peça, sempre com registro no histórico.



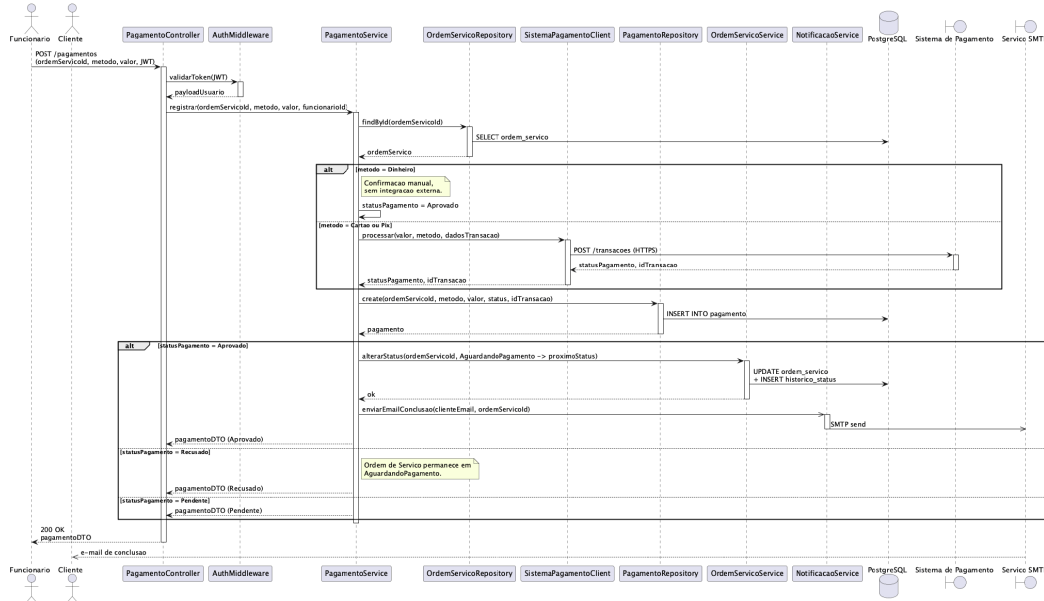
3.4.8 Realização de UC-08 — Registrar Uso de Peças

O CatalogoService localiza a peça (PecaRepository), aplica a regra verificarDisponibilidade e, havendo saldo, executa debitarEstoque e vincula o uso à Ordem de Serviço (ItemOrcamentoRepository); sem saldo, a operação é bloqueada.



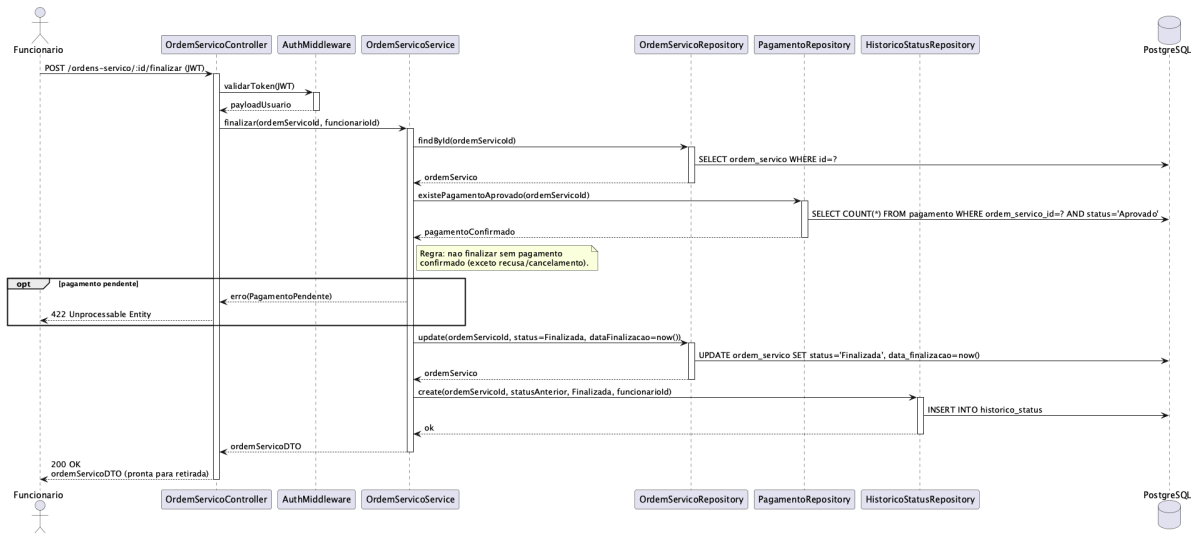
3.4.9 Realização de UC-09 — Registrar Pagamento

Envolve o PagamentoController, o PagamentoService, o SistemaPagamentoClient, o OrdemServicoService (para a transição de status) e o NotificacaoService. O fluxo bifurca em três caminhos no fragmento alt: pagamento em dinheiro (confirmação manual), pagamento por cartão ou PIX (via Sistema de Pagamento externo) e os três possíveis status retornados (Aprovado, Recusado, Pendente). Quando aprovado, o sistema dispara uma notificação por SMTP para o Cliente, sem bloquear a resposta ao funcionário.



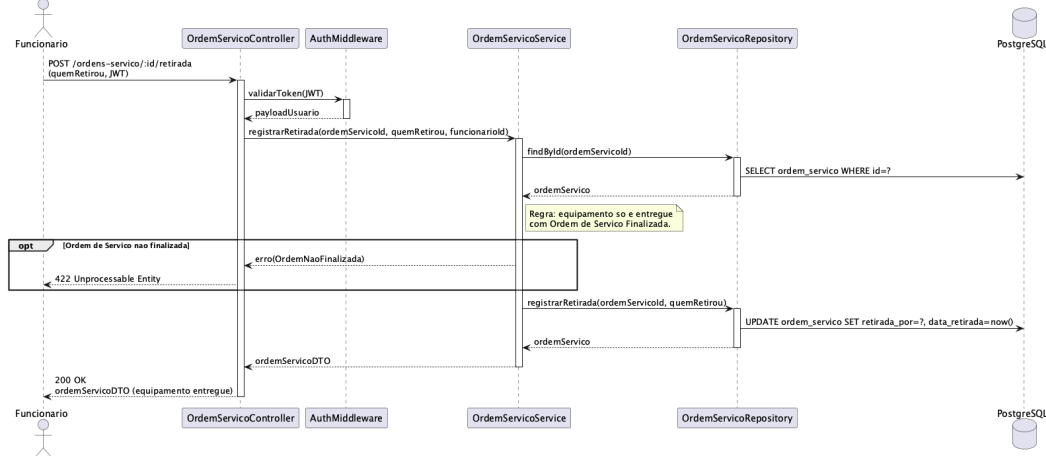
3.4.10 Realização de UC-10 — Finalizar Ordem de Serviço

O OrdemServicoService verifica, via PagamentoRepository, a existência de pagamento aprovado antes de alterar o status para Finalizada e registrar a transição; sem pagamento confirmado, a finalização é impedida.



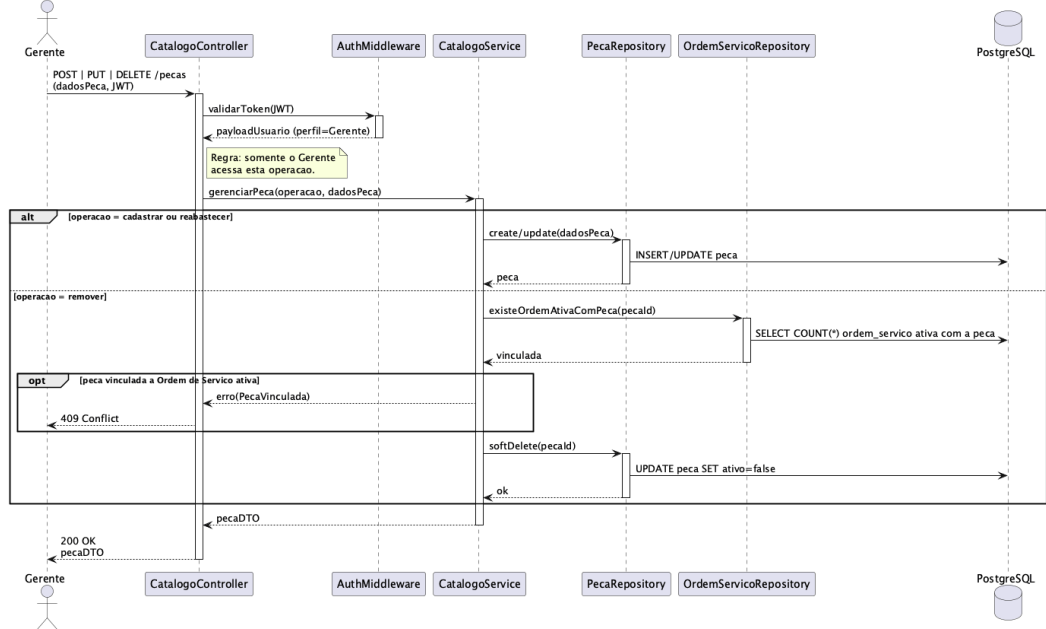
3.4.11 Realização de UC-11 — Registrar Retirada do Equipamento

Após confirmar que a Ordem de Serviço está Finalizada, o OrdemServiceService registra a retirada do equipamento (responsável e data) via OrdemServiceRepository, encerrando o ciclo de atendimento.



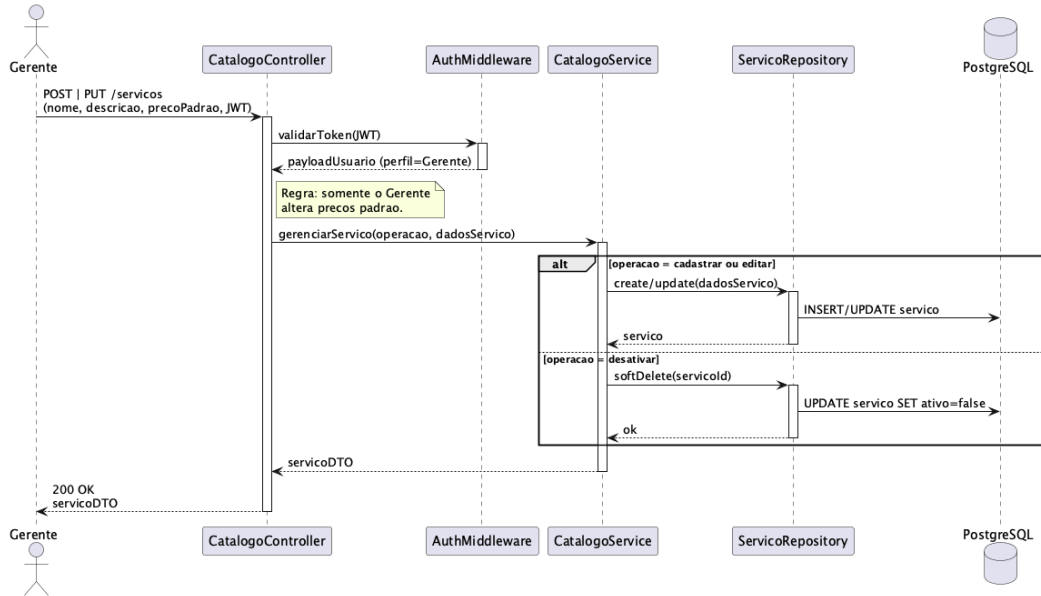
3.4.12 Realização de UC-12 — Gerenciar Peças e Estoque

Operação restrita ao Gerente: o CatalogoService cadastra, reabastece ou desativa peças (PecaRepository), bloqueando a remoção de peças vinculadas a Ordens de Serviço ativas.



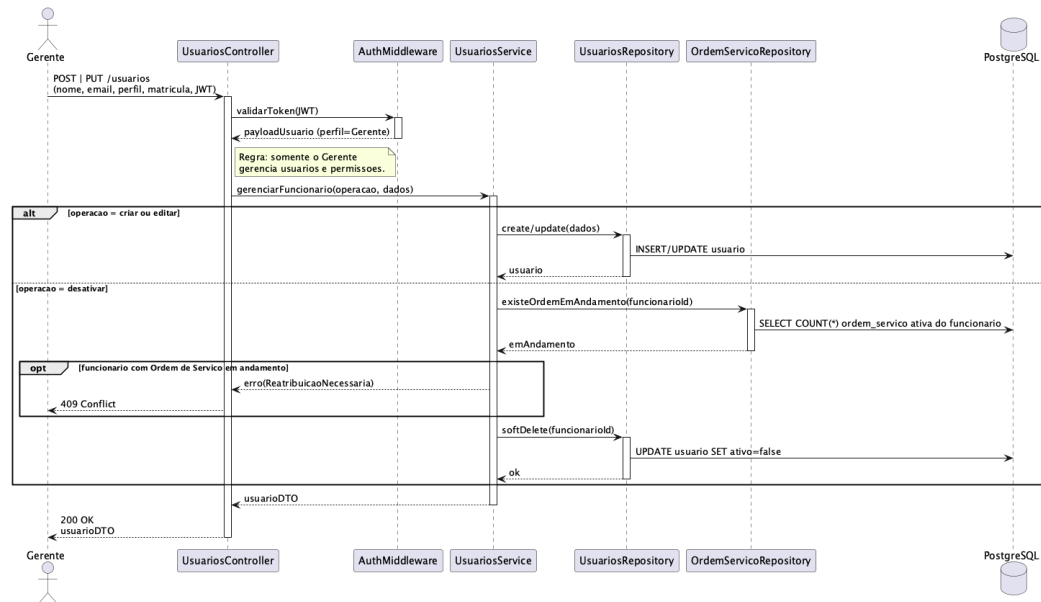
3.4.13 Realização de UC-13 — Gerenciar Serviços e Preços

Operação restrita ao Gerente: o CatalogoService cadastra, edita ou desativa serviços e seus preços padrão (ServicoRepository), mantendo o catálogo coerente para uso nos orçamentos.



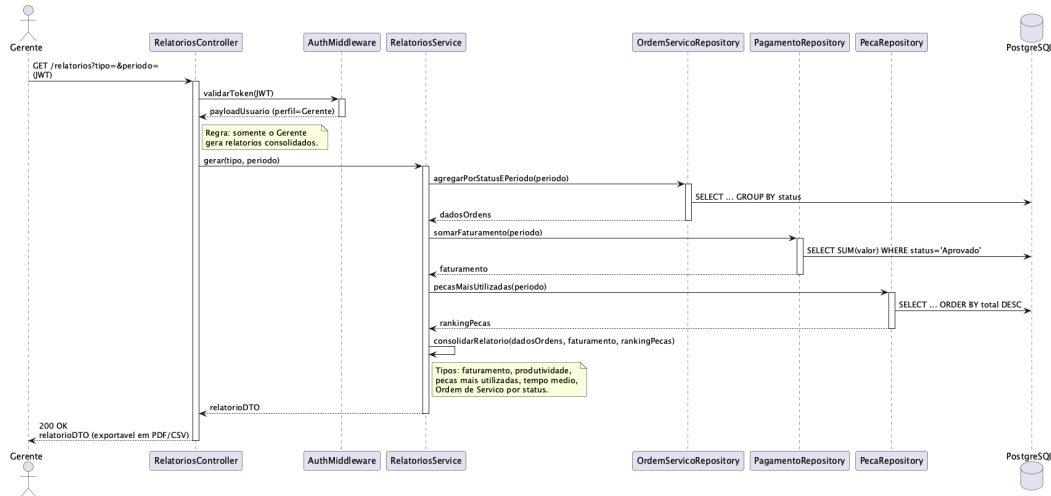
3.4.14 Realização de UC-14 — Gerenciar Funcionários

Operação restrita ao Gerente: o UsuariosService cria, edita ou desativa funcionários (UsuariosRepository), exigindo reatribuição prévia quando o usuário possui Ordem de Serviço em andamento.



3.4.15 Realização de UC-15 — Gerar Relatórios Gerenciais

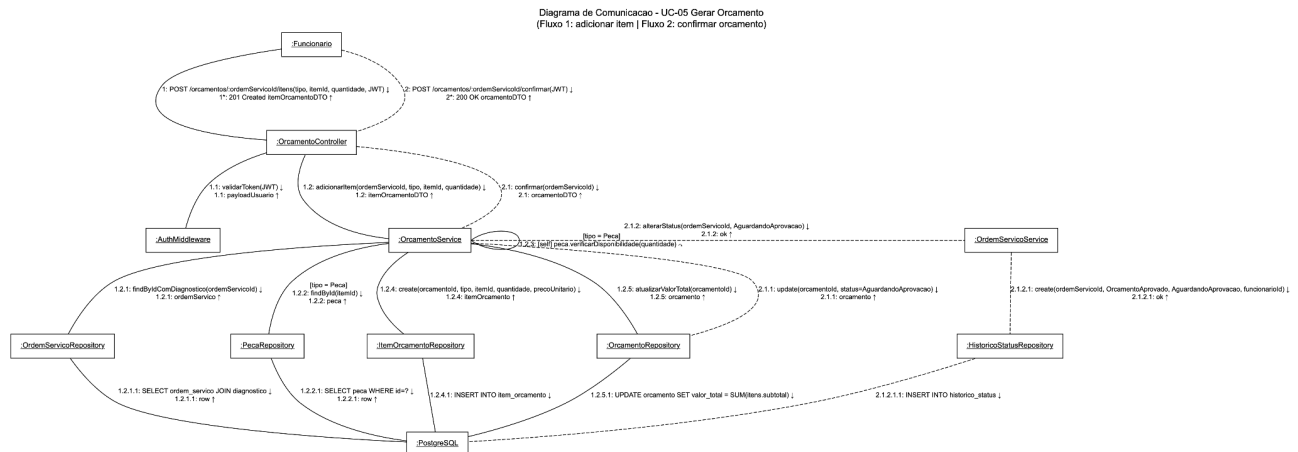
Operação restrita ao Gerente: o RelatoriosService consolida dados de múltiplos repositórios (Ordens de Serviço por status, faturamento, peças mais utilizadas) e devolve o relatório, exportável em PDF ou CSV.



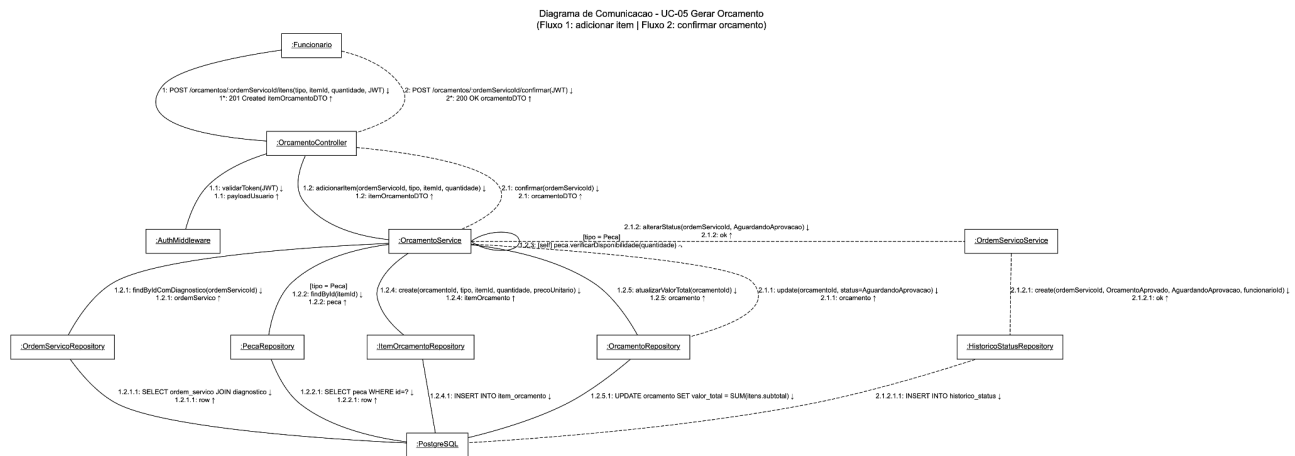
3.5 Diagramas de Comunicação

Diferentemente do diagrama de sequência, que evidencia o eixo temporal das mensagens, o diagrama de comunicação enfatiza os relacionamentos entre os objetos colaboradores: cada nó representa uma instância e cada aresta carrega tanto a chamada quanto o retorno, identificados por uma numeração hierárquica (1, 1.1, 1.2, 1.2.1...). As duas notações são equivalentes em conteúdo, mas complementares em foco. A sequência mostra "quando", a comunicação mostra "quem fala com quem". Foram modelados três diagramas, um para cada um dos 3 seguintes casos de uso: UC-03, UC-05 e UC-09. Em todos os diagramas, arestas tracejadas indicam ramos condicionais ou fluxos separados, e o símbolo $\cup$ marca auto-mensagens (objeto enviando mensagem a si mesmo).

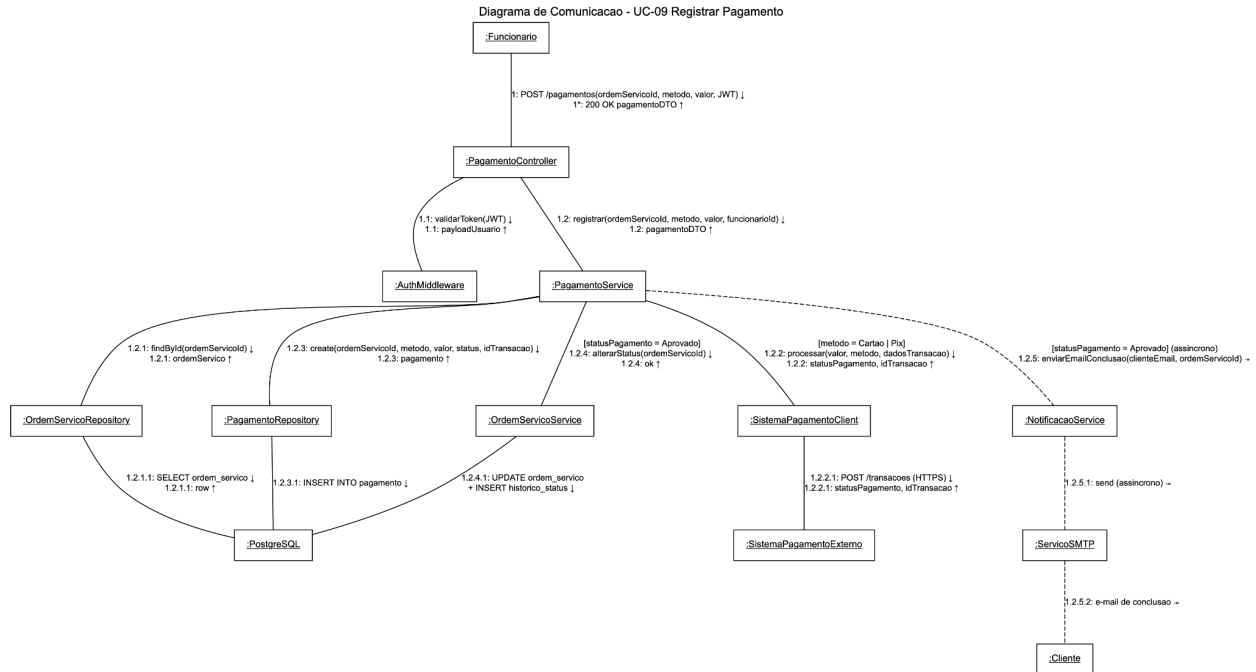
No fluxo de abertura, o *OrdemServicoController* recebe a requisição do Funcionário (mensagem 1), valida o token via *AuthMiddleware* (1.1) e delega ao *OrdemServicoService* (1.2). O serviço executa três validações em sequência, cada uma em um repositório distinto, existência do cliente (1.2.1), do equipamento (1.2.2) e ausência de Ordem de Serviço ativa para esse equipamento (1.2.3) antes de persistir a nova Ordem (1.2.4) e registrar o evento no *HistoricoStatusRepository* (1.2.5). Observa-se que *OrdemServicoRepository* é invocado duas vezes com finalidades diferentes (consulta e criação), o que evidencia o reúso do repositório como ponto de acesso único à tabela.



Já o abaixo (para o UC-05) é o único diagrama com dois fluxos numerados independentemente, refletindo a estrutura em dois fragmentos do diagrama de sequência correspondente. O Fluxo 1 (mensagens 1.x, arestas sólidas) modela a adição iterativa de um item ao orçamento: validação da Ordem de Serviço com diagnóstico (1.2.1), consulta da peça quando aplicável (1.2.2), auto-mensagem verificarDisponibilidade no próprio OrcamentoService (1.2.3, marcada com $\cup$) que aplica a regra de bloqueio por falta de estoque, persistência do item (1.2.4) e recálculo do valor total do orçamento (1.2.5). O Fluxo 2 (mensagens 2.x, arestas tracejadas) representa a confirmação final, que atualiza o status do orçamento (2.1.1) e dispara, via OrdemServicoService, a transição de estado da Ordem com registro no histórico (2.1.2 $\rightarrow$ 2.1.2.1 $\rightarrow$ 2.1.2.1.1).



Por fim, o diagrama mais rico em colaborações, pois UC-09 atravessa todas as camadas e aciona dois sistemas externos. Após autenticação (1.1) e delegação ao PagamentoService (1.2), o serviço consulta a Ordem de Serviço (1.2.1) e bifurca o fluxo segundo o método de pagamento: para cartão ou Pix, invoca o SistemaPagamentoClient (1.2.2), que atua como anti-corruption layer e encapsula a chamada HTTPS ao SistemaPagamentoExterno (1.2.2.1); para dinheiro, a confirmação é manual e essa cadeia é suprimida. Em seguida, persiste o Pagamento (1.2.3) e, somente se aprovado, dispara a transição de estado da Ordem de Serviço (1.2.4) e o envio assíncrono de notificação por e-mail (1.2.5 $\rightarrow$ 1.2.5.1 $\rightarrow$ 1.2.5.2, representadas com setas de mensagem assíncrona $\Rightarrow$ e arestas tracejadas), de modo que a resposta ao Funcionário não fique bloqueada pela entrega do SMTP.



3.6 Diagrama de Estados

A *OrdemServico* é a entidade central do domínio e percorre um ciclo de vida bem-definido, governado por regras de negócio e pelos casos de uso operacionais. Optou-se por modelar **um diagrama de estados** focado nessa entidade, contemplando os 12 estados definidos no plano de projeto e no enum *StatusOrdemServico* da Seção 3.3.

3.6.1 Estados e organização

Os 12 estados são: **Aberta, Em Diagnóstico, Aguardando Orçamento, Aguardando Aprovação, Orçamento Aprovado, Orçamento Recusado, Em Manutenção, Aguardando Peça, Serviço Concluído, Aguardando Pagamento, Finalizada e Cancelada.**

Para organizar a leitura do diagrama, cinco desses estados, todos relativos a atividades **internas** da oficina, foram agrupados em um super-estado chamado **`EmAtendimento`**: *EmDiagnostico*, *AguardandoOrçamento*, *EmManutencao*, *AguardandoPeça* e *ServicoConcluido*. Esse super-estado é visitado em dois momentos distintos do ciclo de vida:

- Antes da aprovação do orçamento (diagnóstico → montagem do orçamento).
- Após a aprovação (manutenção → conclusão do serviço).

Os demais estados, *Aberta*, *AguardandoAprovacao*, *OrcamentoAprovado*, *OrcamentoRecusado*, *AguardandoPagamento*, *Finalizada* e *Cancelada*, permanecem no nível superior do diagrama, pois representam pontos de decisão do cliente ou estados de fronteira de ciclo.

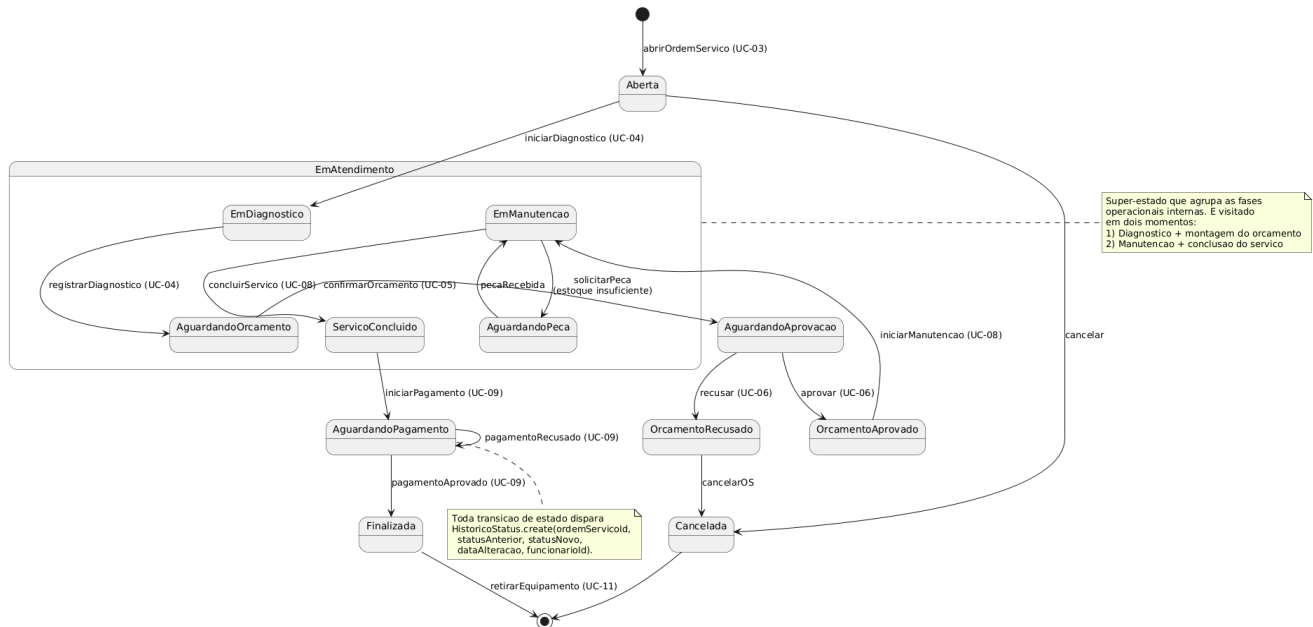
3.6.2 Transições e gatilhos

O estado inicial é **Aberta**, alcançado a partir do caso de uso UC-03 (Abrir Ordem de Serviço). Os estados terminais (que levam a [*]) são **Finalizada** (após UC-11 - Registrar Retirada do Equipamento) e **Cancelada**. As principais transições são:

- **Aberta** → **EmDiagnostico** em UC-04 (iniciar diagnóstico).
- **EmDiagnostico** → **AguardandoOrcamento** em UC-04 (registrar diagnóstico).
- **AguardandoOrcamento** → **AguardandoAprovacao** em UC-05 (confirmar orçamento).
- **AguardandoAprovacao** → **OrcamentoAprovado** / **OrcamentoRecusado** em UC-06.
- **OrcamentoAprovado** → **EmManutencao** em UC-08 (iniciar manutenção).
- **EmManutencao** ↔ **AguardandoPeca**: ciclo de espera por peça quando o estoque é insuficiente.
- **EmManutencao** → **ServicoConcluido** em UC-08 (concluir serviço).
- **ServicoConcluido** → **AguardandoPagamento** em UC-09.
- **AguardandoPagamento** → **Finalizada** quando o pagamento é aprovado; permanece em **AguardandoPagamento** (auto-loop) se recusado.
- **OrcamentoRecusado** → **Cancelada** quando o cliente opta por encerrar a Ordem de Serviço.

3.6.3 Registro histórico

Por via de regra, **toda transição de estado dispara a criação de um registro em `HistoricoStatus`** (ver Seção 3.3), contendo `statusAnterior`, `statusNovo`, `dataAlteracao` e `funcionarioId`. Essa regra está documentada como nota no diagrama para não poluir as transições individuais.



4. Modelos de Dados

Esta seção descreve a persistência do Reparo Tech: o esquema relacional no PostgreSQL (tabelas, colunas, chaves primárias e estrangeiras, índices), a estratégia de mapeamento objeto-relacional adotada (Prisma) e o diagrama Entidade-Relacionamento.

4.1 Esquema relacional

São 11 tabelas. Para a hierarquia de **Usuário**, foi adotada uma única tabela **usuario** discriminada pela coluna **tipo** (FUNCIONARIO ou GERENTE). Essa escolha simplifica consultas de autenticação e o mapeamento via Prisma, relevante porque Funcionário e Gerente compartilham a maior parte dos atributos e o Gerente herda todas as operações de Funcionário (ver Seção 3.3).

4.1.1 Índices não triviais

Além dos índices implícitos de PK e UNIQUE:

- **ordem_servico(status)**: otimiza filtros do dashboard operacional e dos relatórios (UC-15).
- **ordem_servico(cliente_id)**, **ordem_servico(equipamento_id)**: consultas por histórico.
- **ordem_servico(data_abertura)**: relatórios por período.
- **historico_status(ordem_servico_id, data_alteracao)**: reconstrução cronológica.
- **pagamento(ordem_servico_id)**: somatório por Ordem de Serviço.

4.1.2 Restrição de integridade XOR em item_orcamento

Como um item de orçamento referencia um Serviço **ou** uma Peça (mas nunca ambos), aplica-se uma CHECK constraint:

```
CHECK ( (servico_id IS NOT NULL AND peca_id IS NULL) OR (servico_id IS NULL AND peca_id IS NOT NULL) )
```

A coluna **tipo** (TipoItem) reforça semanticamente o XOR e é usada pela camada de aplicação.

4.2 Mapeamento objeto-relacional (Prisma)

O ORM adotado é o **Prisma**, escolhido por se integrar nativamente ao stack Next.js (TypeScript), oferecer migrações versionadas e gerar um cliente *type-safe* a partir do `schema.prisma`. Os principais elementos do mapeamento são:

- **Modelos**: cada tabela tem um `model` correspondente no `schema.prisma`. Nomes em PascalCase (`Usuario`, `OrdemServico`, `ItemOrcamento`...) mapeiam para os nomes de tabela snake_case via `@@map`.
- **Enums**: `StatusOrdemServico`, `StatusOrcamento`, `StatusPagamento`, `MetodoPagamento`, `TipoItem` e `TipoUsuario` são declarados como `enum` no Prisma e materializados como tipos ENUM nativos do PostgreSQL.

- **Hierarquia de `Usuario`:** segue um único model `Usuario` com campo `tipo: TipoUsuario`. A diferenciação entre Funcionário e Gerente é feita na camada de aplicação (por exemplo, em `guards` do `AuthModule` que verificam `tipo === 'GERENTE'`).

- **Relacionamentos:** declarados via `@relation` com `fields` e `references`. Relações 1:0..1 (`OrdemServico` — ``Diagnosticos`` e ``OrdemServico`` — `Orcamento`) são modeladas com a FK no lado dependente marcada como `@unique`.

- **XOR de `ItemOrcamento`:** dado que o Prisma não expressa CHECK constraints nativamente, a restrição é adicionada via migração SQL bruta (`prisma migrate`) e validada também na camada de serviço (`OrcamentoService`).

- **Soft-delete:** colunas `ativo`: `Boolean @default(true)` em `usuario`, `servico` e `peca`. As consultas padrão filtram por `ativo: true`.

- **Auditoria temporal:** colunas `data_cadastro`, `data_abertura`, `data_alteracao`, etc., recebem `@default(now())` quando aplicável.

- **Migrações:** versionadas em `prisma/migrations/`, com nomes descritivos.

4.3 Diagrama Entidade-Relacionamento

O diagrama abaixo formaliza graficamente as tabelas, chaves e cardinalidades descritas em 4.1.

